

基于成分数据建模与多模型融合的古 代玻璃制品成分分析与鉴别

赵宇翔

李传浩

马文杰

2026 年 9 月 14 日

摘要

本文针对古代玻璃制品的成分分析与鉴别问题，围绕成分数据定和约束、检出限缺失与风化扰动三大难点，建立了“数据预处理—风化规律建模—分类聚类—未知样本判别”的完整框架，并对结论进行了稳健性检验。

针对问题一：以 85% ~ 105% 累加和标准筛选有效数据，对中缺失成分采用 **SVR** 插补，经采样点聚合与 **ALR / CLR** 对数比变换消除成分伪相关。运用卡方检验与蒙特卡洛置换检验方法，结合 **Spearman**、**Kendall** 秩相关，得出风化与玻璃类型显著相关，与纹饰、颜色无显著关联的结论；运用 **Mann-Whitney U** 检验与 **Cohen's d** 效应量，结果显示高钾风化后 SiO_2 富集、铅钡 SiO_2 流失。进一步构建风化程度回归并估计反应比例，以此来反推风化前成分，并运用虚拟变量回归对照验证。

针对问题二：以随机森林特征重要性筛选前 7 位核心成分，构建逻辑回归、随机森林与深度受限决策树三判别模型，并用分层 5 折交叉验证评估三模型泛化能力；两类玻璃经 **K-means++** 聚类得高钾 $K = 4$ 、铅钡 $K = 6$ 个亚类，ARI 与 $\pm 5\%$ 、 $\pm 10\%$ 的扰动检验及 **Ward** 对照验证了划分的合理性与稳健性。

针对问题三：运用问题二的判别底座对 8 件未知样品判别，风化样本经假设-验证还原风化前成分后再判别，判定 A1、A6、A7 为高钾，A2、A3、A4、A5、A8 为铅钡； $\pm 5\%$ 、 $\pm 10\%$ 扰动下 100 次判别类别翻转率均为 0，结果稳健可靠。

关键词：古代玻璃；成分数据分析；SVR 插补；风化前成分反推；随机森林；K-means++；敏感性分析

1 问题重述

古玻璃是丝绸之路中西贸易与文化交流的重要物证。我国本土古玻璃借鉴域外技术，外观与外来玻璃相近，但化学成分存在明显区别。玻璃基体成分为二氧化硅，炼制时添加不同助熔剂形成两大类：草木灰作为助熔剂得到高钾玻璃，氧化钾含量突出；铅钡矿石作为助熔剂得到铅钡玻璃，氧化铅、氧化钡含量较高。文物埋藏过程中易发生风化，元素交换会改变成分占比，干扰类型识别。

题目提供三套数据集：文物基础信息、已知类别玻璃化学成分、未知类别玻璃化学成分。规定成分总和在 85% ~ 105% 视为有效数据。需要完成三项任务：分析表面风化与玻璃类型、纹饰、颜色的关系，统计风化带来的成分变化并预测风化前化学成分；挖掘高钾、铅钡玻璃分类规律，在两类内部开展亚类划分并检验合理性与敏感性；判别未知玻璃文物类别并做敏感性分析。

2 问题分析

本题主要难点来自风化扰动、成分缺失以及成分数据固有的定和约束，需要完成统计关联、成分预测、分层分类与未知样本判别。

问题 1 属于关联分析与预测任务：探究风化与文物属性之间的分布关系；分玻璃类型对比风化-未风化样本的化学成分差异；利用同文物风化、未风化配对样本，建模反推风化前原始成分。

问题 2 为分层分类任务：基于化学成分提炼高钾、铅钡的大类判别规则；分别在两类内部做亚类划分，通过稳定性检验与参数扰动分析结果的合理性与敏感性。

问题 3 是模型应用任务：将问题 2 建立的判别模型用于未知样本识别，结合风化状态，分析成分扰动对判别结果的影响，评估模型鲁棒性。

3 数据预处理

本文数据集为典型氧化物成分数据，受定和闭合约束与仪器检出限影响，普遍存在成分未检出、成分总和偏离 100% 的数据失真问题。为消除成分数据单形空间伪相关性、适配常规统计与机器学习建模，本文参照已有古玻璃数据处理范式（彭丽红等，2024；侯兴汉等，2026）^[1-2]开展系统化预处理。首先通过正则解析拆分原始复合采样点字段，完成多表属性关联，以成分累加和 S_i 作为有效性判定依据。

3.1 累加和有效性筛选

$$S_i = \sum_{d=1}^D x_{id}$$

其中 S_i 为第 i 个采样点的成分累加和， x_{id} 为该采样点第 d 种成分的质量分数， D 为成分总数。参照彭丽红等（2024），限定化学成分比例累加和在 85% ~ 105% 才可视为有效数据，以 $V_i = \mathbf{1}\{85\% \leq S_i \leq 105\%\}$ 标记样本有效性，剔除 2 条无效采样数据，保留 67 条已知、8 条未知有效样本。

3.2 缺失率统计与缺失机制判别

基于缺失率

$$\rho_d = \frac{1}{n} \sum_{i=1}^n \mathbf{1}\{x_{id} \text{ 未检出}\} \times 100\%$$

其中 ρ_d 为第 d 种成分的缺失率， n 为样本总数， $\mathbf{1}\{\cdot\}$ 为示性函数，条件成立时取 1、否则取 0。将 14 种成分划分为高、中、低缺失三档，区分非随机缺失（MNAR）检出限缺失与可插补缺失，高缺失成分仅保留二值检出特征。

3.2.1 SVR 插补

针对中缺失成分，本文参照侯兴汉等（2026）提出一种基于滑动窗口的支持向量回归（SVR）插补方法，通过分阶段网格搜索与 5 折交叉验证优化模型超参数的思路，将之改造适配截面数据，拟合高斯核 SVR 模型

$$\arg \min_{\mathbf{w}, b, \xi, \xi^*} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{j=1}^m (\xi_j + \xi_j^*)$$

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$$

其中 $\mathbf{w}$ 为超平面法向量, b 为偏置项, ξ_j 、 ξ_j^* 为第 j 个训练样本的上、下松弛变量, C 为惩罚系数, m 为参与训练的样本数; $K(\mathbf{x}_i, \mathbf{x}_j)$ 为样本 $\mathbf{x}_i$ 与 $\mathbf{x}_j$ 之间的高斯核函数值, γ 为核宽度参数, $\|\cdot\|$ 表示欧氏范数。仅对 R^2 较高、关联可信的 K_2O 、 PbO 、 BaO 开展数值插补, 其余中缺失成分因拟合效果差仅保留二值特征, 保证插补结果稳健无畸变。

3.2.2 采样点聚合

随后对同文物多采样点聚合取均值

$$\bar{x}_d^{(k)} = \frac{1}{n_k} \sum_{j=1}^{n_k} x_{jd}^{(k)}$$

其中 $\bar{x}_d^{(k)}$ 为第 k 件文物第 d 种成分的聚合均值, n_k 为该文物的采样点个数, $x_{jd}^{(k)}$ 为其第 j 个采样点的成分含量。结合未风化点数据补全样本, 二次校验闭合度后得到 56 组有效文物级样本, 满足建模独立性要求。

3.2.3 对数比变换

本文参照钱鸿羽等 (2024)^[3] 为了消除定和约束的影响, 通过计算各个组别所占成分比例与其均值的比值的对数, 将成分数据从单形空间变换到欧式空间, 从而能够应用经典统计方法解决成分数据问题的处理方法, 对 8 组可靠成分开展对数比变换。先采用极小值替换规避对数无定义问题, 再分别构建以 SiO_2 为基准的加性对数比 (ALR)

$$y_{i,d} = \ln \frac{x_{i,d}}{x_{i,\text{ref}}}$$

其中 $y_{i,d}$ 为第 i 个样本第 d 种成分的 ALR 变换值, $x_{i,d}$ 为变换前的成分含量, $x_{i,\text{ref}}$ 为基准成分 SiO_2 的含量。与基于几何均值的中心对数比 (CLR)

$$g_i = \left(\prod_{d=1}^{D'} x_{i,d} \right)^{1/D'}, \quad y_{i,d} = \ln \frac{x_{i,d}}{g_i}$$

其中 g_i 为第 i 个样本各成分含量的几何均值, D' 为参与对数比变换的成分数, $y_{i,d}$ 为相应的 CLR 变换值。两套变换互补使用, 彻底消除成分伪相关, 最终得到适配统计检验、分类判别与回归预测的标准化数据集。

3.3 实践效果

最终数据按用途保存为相互配套的多个口径: 保留 NaN 版用于统计检验; 填 0 版用于累加和与闭合度复查; SVR 插补版仅对 K_2O 、 PbO 、 BaO 三列补齐缺失值, 供需要完整数值的判别与预测建模使

用；被判定为低于检出限或数值不可靠的成分不进入任何数值变换，仅保留检出 / 未检出二值标志；对数比变换列在低缺失与已插补的 8 种子成分上生成。样本层级上需区分两级有效性：采样点级在 3.2 节按 85% ~ 105% 剔除 2 条后余 67 条；文物级 58 件全量样本经聚合后做闭合度复查，保留 56 件。

4 模型假设

本文做出如下模型假设：未风化文物以及文物未风化部位的检测成分可以近似作为玻璃风化前的原始组分（熊强等，2025），高钾、铅钡玻璃类别在风化前后维持不变；为避免风化引起的化学成分含量变化对判定玻璃体系的影响，从样本数据中选取无风化样本作为数据拟合的内容（钱鸿羽等，2024）；水质数据缺失一般包括完全随机缺失、随机缺失和非随机缺失（侯兴汉等，2026），未检出成分视作低于仪器检出限而非随机缺失；同一类型玻璃发生风化时，由于在化学反应中涉及的化学成分是成比例的（钱鸿羽等，2024），各成分迁移、反应的比例具备相对稳定性，可依托各成分相对于 SiO_2 的反应比例描述风化行为；风化程度以 0 ~ 1 区间连续变量量化（钱鸿羽等，2024）；不同文物样本相互独立，同一文物的多个采样点经聚合后视为单个独立样本，可代表该文物整体成分特征（课件，第 8 讲 - 独立性）。

5 问题 1 模型

5.1 表面风化与玻璃类型、纹饰、颜色的关系

5.1.1 模型讲解

彭丽红等（2024）在同类古玻璃风化研究中运用卡方独立性检验来探讨影响玻璃文物表面风化的因素，并指出进行独立性检验时，若列联表中有单元格频数为零，取 Fisher 精确概率检验结果，否则取卡方检验结果。本文纹饰、颜色的列联表存在零频数与期望频数过小的问题，因此采用蒙特卡洛置换检验作为 Fisher 精确检验的扩展方案；同时参照熊强等（2025）^[4]为更精确地量化各因子与风化情况的相关性，采用 Spearman 秩相关系数来分析、刘振文等（2025）^[5]使用 Kendall tau 系数及其 P 值探究之间的相关性和差异性的思路，开展非参数关联分析，实现多方法交叉验证。

5.1.2 卡方独立性检验与蒙特卡洛置换检验

设属性共有 L 个类别，风化状态分为无风化与风化两类。记第 ℓ 类属性、第 c 类风化的观测频数为 $n_{\ell c}$ ($\ell = 1, \dots, L; c = 1, 2$)，行合计、列合计与总样本数分别为

$$n_{\ell.} = \sum_{c=1}^2 n_{\ell c}, \quad n_{.c} = \sum_{\ell=1}^L n_{\ell c}, \quad n = \sum_{\ell=1}^L \sum_{c=1}^2 n_{\ell c}$$

其中 n 表示参与检验的文物样本数。卡方独立性检验的原假设与备择假设为

$$H_0 : \text{属性与表面风化相互独立}, \quad H_1 : \text{二者不独立}.$$

在 H_0 成立时，格 (ℓ, c) 的期望频数为 $E_{\ell c} = n_{\ell.} n_{.c} / n$ ，据此构造卡方统计量

$$\chi^2 = \sum_{\ell=1}^L \sum_{c=1}^2 \frac{(n_{\ell c} - E_{\ell c})^2}{E_{\ell c}}$$

其自由度 $\nu = (L - 1)(2 - 1) = L - 1$ 。当各格期望频数均不小于 5 时 χ^2 渐近服从 $\chi^2(\nu)$ ，P 值取 $P = \Pr(\chi^2(\nu) \geq \chi_{\text{obs}}^2)$ ，其中 χ_{obs}^2 为观测样本的卡方统计量，显著性水平取 $\alpha = 0.05$ 。纹饰、颜色系列联表存在期望频数过小 ($E_{\ell c} < 5$) 甚至为零的格，卡方近似不再可靠，改用蒙特卡洛置换检验：固定属性类别与样本量，将风化标签随机重排 $N_{\text{perm}} = 5000$ 次并每次重算卡方统计量 χ_t^2 ($t = 1, \dots, N_{\text{perm}}$)，P 值取

$$P_{\text{perm}} = \frac{\sum_{t=1}^{N_{\text{perm}}} \mathbb{I}\{\chi_t^2 \geq \chi_{\text{obs}}^2\} + 1}{N_{\text{perm}} + 1},$$

分子分母同加 1 以避免 $P = 0$ 的伪精确；该检验可视为费希尔精确检验对 $L \times 2$ 列联表的推广。

5.1.3 Spearman 与 Kendall 秩相关

为刻画关联方向并对检验结论交叉验证，进一步计算 Spearman 秩相关 ρ 与 Kendall 秩相关 τ ：将属性类别与风化状态分别编码后再转秩，设第 i 个样本的属性秩与风化秩为 u_i 、 v_i （并列取平均秩， $i = 1, \dots, n$ ），则 ρ 为秩变量 u_i 、 v_i 的皮尔逊相关系数

$$\rho = \frac{\sum_{i=1}^n (u_i - \bar{u})(v_i - \bar{v})}{\sqrt{\sum_{i=1}^n (u_i - \bar{u})^2} \sqrt{\sum_{i=1}^n (v_i - \bar{v})^2}},$$

其中 $\bar{u} = \frac{1}{n} \sum_{i=1}^n u_i$ 、 $\bar{v} = \frac{1}{n} \sum_{i=1}^n v_i$ ；记全部 $\frac{1}{2}n(n-1)$ 个样本对中一致对与不一致对的数目为 n_c 、 n_d ，其中并列对按 Kendall τ_b 修正，则

$$\tau = \frac{2(n_c - n_d)}{n(n-1)}.$$

ρ 与 τ 均取值于 $[-1, 1]$ ，其正负与大小反映关联方向与强弱，与卡方检验共同支撑风化影响因素的判断。

5.1.4 实践效果

基于 58 件文物级样本开展计算，颜色分析剔除 4 条颜色缺失样本并归并色系，所得统计结果如表 1 所示。

表 1: 表面风化与玻璃类型、纹饰、颜色关联性的检验结果

属性	卡方统计量	P 值	Spearman ρ	Kendall τ	结论
类型	5.452	0.0195	-0.344**	-0.344**	显著相关
纹饰	4.957	0.106*	+0.037	+0.036	不显著
颜色	2.126	0.659*	-0.020	-0.020	不显著

注：* 为蒙特卡洛置换检验 p 值；** 表示 $p < 0.01$ 。

多种检验得到一致结论：表面风化与玻璃类型显著相关，铅钡玻璃风化占比 70%，高钾玻璃风化占比仅 33%；风化与纹饰、颜色不存在显著关联。

5.2 风化前后化学成分统计规律

5.2.1 模型讲解

钱鸿羽等（2024）对化学成分含量进行正态性检验，发现总体不符合正态分布，因此选择非参数检验中的独立样本 Mann-Whitney U 检验，按照 Cohen's d 系数排序分析具有显著性差异的化学成分。本文数据集各成分同样偏离正态分布，直接复用该检验框架；同时参考熊强等（2025）将高钾玻璃分为未风化和风化两类，铅钡玻璃分为未风化、风化和严重风化三类的处理方式，分玻璃类型对比风化、未风化样本的成分统计特征。具体以文物级主表（51 件）为数据口径、按类型 $\times$ 风化分组；因部分成分存在未检出，各组各成分的有效样本数以实际检出数为准（不足 2 个时不做检验，如高钾玻璃中多数风化样本未检出的成分）。

5.2.2 Mann-Whitney U 检验

记类型内第 d 种成分为检验对象，无风化组、风化组的有效样本数与样本均值分别为 n_0 、 n_1 与 $\bar{x}_{0,d}$ 、 $\bar{x}_{1,d}$ 。Mann-Whitney U 检验的原假设与备择假设为

$$H_0: \text{两组含量分布相同}, \quad H_1: \text{两组含量分布不同}$$

将两组共 $n_0 + n_1$ 个观测合并后按升序编号取秩，设风化组的秩和为 R_1 ，则统计量

$$U = n_0 n_1 + \frac{n_1(n_1 + 1)}{2} - R_1$$

其含义等价于风化组中每个观测大于无风化组观测的次数之和。在 H_0 下 U 的分布仅取决于 n_0 、 n_1 ，其中小样本取精确分布，大样本可用正态近似，据此计算双尾 P 值；显著性水平取 $\alpha = 0.05$ ， $P < \alpha$ 时认为该成分含量在风化前后存在显著差异。

5.2.3 Cohen's d 效应量

进一步以 Cohen's d 效应量刻画差异大小。组内样本方差与合并标准差分别为

$$s_g^2 = \frac{1}{n_g - 1} \sum_{i=1}^{n_g} (x_{g,i} - \bar{x}_{g,d})^2, \quad g = 0, 1$$

$$s_p = \sqrt{\frac{(n_0 - 1)s_0^2 + (n_1 - 1)s_1^2}{n_0 + n_1 - 2}}$$

其中 $x_{g,i}$ 为该成分在组 g 中的第 i 个观测， n_g 为组 g 的有效样本数，效应量取

$$d = \frac{\bar{x}_{1,d} - \bar{x}_{0,d}}{s_p},$$

$d > 0$ 表示风化后含量升高、 $d < 0$ 表示下降， $|d|$ 约 0.2, 0.5, 0.8 分别对应小、中、大效应。

5.2.4 实践效果

对两类玻璃的各成分逐对检验，即可结合显著性、 d 的符号与大小归纳风化前后的含量统计规律。高钾玻璃中，风化后 SiO_2 均值由 66.69% 上升至 93.96% ($P < 0.001$, Cohen's $d = 4.80$)， K_2O 、 CaO 、 Al_2O_3 、 Fe_2O_3 均出现显著下降。铅钡玻璃表现出相反变化趋势： SiO_2 由 54.93% 下降至 27.73% ($P < 0.001$, $d = -2.45$)， PbO 、 CaO 、 P_2O_5 含量显著升高。高钾玻璃风化过程表现为二氧化硅富集、助熔与碱金属组分流失；铅钡玻璃则发生二氧化硅流失、重金属氧化物相对富集。

5.3 风化前化学成分预测

5.3.1 模型讲解

借鉴钱鸿羽等（2024）的建模思路：构建风化程度多元函数，依据风化过程的成分成比例迁移特性估计各组分反应比例，反算风化损耗量，风化前的数据可以通过从风化后的数据中减去变化来获得，进而还原风化前原始成分；同时引入熊强等（2025）将采样部位的风化程度作为核心解释变量，建立包含定性变量的多元线性回归方程进行分析的回归模型作为对照，校验预测趋势合理性。

建模以有效采样点为单元，记风化样本 i 第 d 种成分的实测含量为 x_{id} ，未风化点及无风化样本的检测值视为风化前的近似基准，风化前成分反推分三步：建立风化程度函数、估计风化反应比例、反算风化损耗量还原原始成分。

5.3.2 风化程度多元回归

第一步，为每个采样点赋风化程度 Y_i ：无风化取 $Y_i = 0$ ，严重风化点取 $Y_i = 1$ ，一般风化点按类型取 $Y_i = 0.8$ 或 $Y_i = 0.75$ 。对类型内选定的关键成分子集 S 拟合风化程度多元线性回归

$$Y_i = \beta_0 + \sum_{d \in S} \beta_d x_{id}$$

回归系数 (β_0, β_d) 由最小二乘法确定，即使残差平方和最小， $\hat{\beta}_0$ 、 $\hat{\beta}_d$ 即其最小二乘估计值

$$(\hat{\beta}_0, \hat{\beta}_d) = \arg \min_i \sum (Y_i - \beta_0 - \sum_{d \in S} \beta_d x_{id})^2$$

R^2 反映所选成分对风化程度的可解释比例。

5.3.3 风化反应比例估计

第二步，估计各成分相对 SiO_2 的风化反应比例。记类型内无风化组、风化组第 d 种成分的均值分别为 $\bar{x}_{0,d}$ 、 $\bar{x}_{1,d}$ （与 5.1、5.2 的 0 为无风化、1 为风化编码一致），风化引起的含量差为 $\Delta_d = \bar{x}_{1,d} - \bar{x}_{0,d}$ ，则反应比例

$$r_d = \frac{\Delta_d}{\Delta_{\text{SiO}_2}}, \quad r_{\text{SiO}_2} = 1$$

$r_d < 0$ 表示风化过程中该成分相对 SiO_2 流失， $r_d > 0$ 表示相对富集。

5.3.4 风化前成分反推

第三步，反推风化前成分。设风化引起的各成分变化按反应比例成比例迁移，即 $\Delta x_d = r_d \Delta x_{\text{SiO}_2}$ ；将回归式写成关于 Δx_{SiO_2} 的线性增量并与风化程度 Y_i 对应，可解出 SiO_2 的变化量，进而还原各成分风化前含量估计 $\hat{x}_{id}$

$$\hat{x}_{id} = x_{id} - \frac{r_d}{A} Y_i, \quad A = \beta_{\text{SiO}_2} + \sum_{d \in S \setminus \{\text{SiO}_2\}} \beta_d r_d$$

其中 A 为反推分母。风化组中 Na_2O 、 PbO 、 BaO 等成分多数样本未检出， r_d 与实测含量不可得，仅对可检出且反应比例有效的成分反推， SnO_2 、 SO_2 等不参与。

5.3.5 标准化偏差检验

预测合理性以标准化偏差检验：设反推所得风化前含量的均值为 $\bar{\hat{x}}_d$ ，与无风化组均值 $\bar{x}_{0,d}$ 、样本标准差 $s_{0,d}$ 比较

$$\frac{|\bar{\hat{x}}_d - \bar{x}_{0,d}|}{s_{0,d}} < 1$$

小于 1 表示反推结果落在无风化组均值一倍标准差之内，预测与未风化样本分布一致。

5.3.6 风化虚拟变量回归对照

作为对照，按熊强等（2025）的风化虚拟变量回归复核趋势方向：对风化显著相关成分 d ，以少量低缺失成分为特征集 $\mathcal{F}_d$ 拟合

$$x_{id} = \alpha_d + \sum_{f \in \mathcal{F}_d} \beta_{df} x_{if} + \lambda_d WG_i, \quad WG_i = \mathbb{I}\{Y_i > 0\}$$

其中 WG_i 为风化虚拟变量， λ_d 即风化对含量的净效应， α_d 为截距项、 β_{df} 为特征 f 的回归系数；将风化样本的 WG 置 0 即得风化前预测，据此与主线 A 的升降方向、量级相互印证。

5.3.7 实践效果

按上述风化程度赋值与关键成分分子集 S ，分别对高钾、铅钡玻璃拟合成风化程度函数，得到

$$Y_K = 0.200 + 0.0063x_{\text{SiO}_2} - 0.0783x_{\text{K}_2\text{O}} + 0.0770x_{\text{CaO}} - 0.0396x_{\text{Al}_2\text{O}_3}$$

其中 Y_K 为高钾玻璃的风化程度函数取值， x_{SiO_2} 、 $x_{\text{K}_2\text{O}}$ 、 x_{CaO} 、 $x_{\text{Al}_2\text{O}_3}$ 为对应成分的实测含量， $R^2 = 0.862$ ；

$$Y_{Pb} = 0.7627 - 0.0156x_{\text{SiO}_2} - 0.1384x_{\text{CaO}} + 0.0476x_{\text{Al}_2\text{O}_3} + 0.0070x_{\text{PbO}} + 0.0544x_{\text{P}_2\text{O}_5}$$

其中 Y_{Pb} 为铅钡玻璃的风化程度函数取值， $R^2 = 0.781$ 。反应比例要利用风化组与未风化组的均值差异估计组分反应比例，高钾体系

$$\text{SiO}_2 : \text{K}_2\text{O} : \text{CaO} : \text{Al}_2\text{O}_3 = 1 : -0.339 : -0.201 : -0.166$$

铅钡体系

$$\text{SiO}_2 : \text{CaO} : \text{Al}_2\text{O}_3 : \text{PbO} : \text{P}_2\text{O}_5 = 1 : -0.044 : 0.027 : -0.605 : -0.137$$

对 4 个高钾风化采样点完成反推得到预测结果, 风化前 SiO_2 预测区间 58.9% ~ 63.3%, K_2O 处于 11.9% ~ 12.3%; 23 个铅钡风化采样点全部预测, 其中严重风化样本 08 的原始 SiO_2 预测值 54.83%, 落在未风化铅钡样本均值的一倍标准差范围内。以标准化偏差作为评价指标, 高钾全部可预测成分、铅钡除去高缺失组分以外的成分标准化偏差均小于 1, 预测结果与未风化样本分布保持一致。构建风化虚拟变量回归对照模型, 高钾 K_2O 模型 $R^2 = 0.945$, 风化虚拟变量系数为 -3.26 ; 铅钡 P_2O_5 模型 $R^2 = 0.842$, 系数为 $+2.96$ 。

然而, 数据集无高钾玻璃严重风化样本, 因此高钾风化程度函数仅包含 $Y_i = 0$ 、 $Y_i = 0.8$ 两档; SnO_2 、 SO_2 、 Na_2O 等高缺失成分大多低于仪器检出限, 不参与风化前成分反推, 无法预测。

6 问题 2 模型

6.1 高钾、铅钡玻璃的分类规律

6.1.1 模型讲解

张星园等 (2023)^[6]通过综合相关性分析研究两种玻璃的分类规律, 发现玻璃类型和二氧化硅、氧化钾、氧化铅和氧化钡的相关系数可达到 0.7。本文先以 Spearman 秩相关刻画类型与各成分的关联强度, 再按刘振文等 (2025) 使用网格寻优对 RF 超参数进行调节, 以此来获得影响样本分类的 7 个重要指标进行数据降维的思路, 以随机森林特征重要性筛选分类核心成分, 同时拟合逻辑回归判别函数 (钱鸿羽等, 2024), 并以深度受限决策树输出可读分类规则 (熊强等, 2025); 各模型准确率均采用分层 5 折交叉验证报告, 标准化参数仅在训练折内估计, 避免数据泄漏 (课件, 第 3 讲 - K 折交叉验证; 课件, 第 3 讲 - 数据泄漏)^[7]。

6.1.2 特征标准化与 Spearman 秩相关

设以预处理后的 56 件有效文物级样本为建模对象, 类型编码为二值标签 y_i (高钾 $y_i = 0$ 、铅钡 $y_i = 1$, 与表 2 注一致); 第 i 个样本第 d 种成分含量 C_{id} (即 $C_{\text{SiO}_2} = C_{i,\text{SiO}_2}$ 一类的含量记号, 化学式下标同 5.3) 尺度差异大, 建模前先做 Z 分数标准化

$$z_{id} = \frac{C_{id} - \bar{C}_d}{s_d}$$

其中 $\bar{C}_d$ 、 s_d 为成分 d 在训练样本上的均值与样本标准差 (仅在训练折内估计, 避免数据泄漏)。先以 Spearman 秩相关刻画类型与各成分的单调关联方向: 将 y_i 与 C_{id} 转为秩 u_i 、 v_i (并列取平均秩), 得相关系数

$$\rho_d = \frac{\sum_i (u_i - \bar{u})(v_i - \bar{v})}{\sqrt{\sum_i (u_i - \bar{u})^2} \sqrt{\sum_i (v_i - \bar{v})^2}}$$

其中 $\bar{u}$ 、 $\bar{v}$ 为属性秩与成分秩的均值； $\rho_d > 0$ 表示该成分随铅钡倾向升高， $\rho_d < 0$ 表示随高钾倾向升高（张星园等，2023）。

判别模型沿三条途径交叉构建并互为印证。

6.1.3 随机森林特征重要性

其一，随机森林以基尼不纯度下降度量特征重要性（刘振文等，2025）：节点 ν 的两类占比为 π_0, π_1 ，不纯度 $G(\nu) = 1 - \sum_{k=0}^1 \pi_k^2$ ，按成分 d 分裂为子节点 ν_L, ν_R 的不纯度下降为

$$\Delta G(\nu) = G(\nu) - \frac{n_{\nu_L}}{n_\nu} G(\nu_L) - \frac{n_{\nu_R}}{n_\nu} G(\nu_R),$$

其中 n_ν 、 n_{ν_L} 、 n_{ν_R} 分别为节点 ν 及其左右子节点的样本数；将其在所有树、所有分裂点上累加并归一化即得重要性 v_d ，据此取累计重要性达 91.6% 的前 7 位成分作为判别核心成分，实现降维并剔除噪声。

6.1.4 逻辑回归判别

其二，逻辑回归以标准化成分的线性组合经 Sigmoid 变换输出铅钡概率（钱鸿羽等，2024）

$$\eta_i = a_0 + \sum_{d=1}^{14} a_d z_{id}, \quad P_i = \Pr(y_i = 1 | z_i) = \frac{1}{1 + e^{-\eta_i}},$$

其中 η_i 为标准化成分的线性组合（对数几率）、 a_0 为截距项；判别系数 a_d （表 2）为正促进判铅钡、为负促进判高钾， $P_i > 0.5$ 判为铅钡。

6.1.5 CART 决策树

其三，深度受限（深度 3）的 CART 决策树同样以基尼下降最大选择划分，叶节点取多数类，输出“ $\text{PbO} \leq 5.46$ 判高钾、 $\text{PbO} > 5.46$ 判铅钡”的可读规则（熊强等，2025）。

6.1.6 分层 5 折交叉验证

三模型的泛化能力均以分层 5 折交叉验证报告：按类别比例分层将 n 个样本分为 5 折，逐折以其余 4 折训练、该折测试，折外准确率

$$\text{Acc} = \frac{1}{n} \sum_{i=1}^n 1\{\hat{y}_i = y_i\}$$

每折类别比例与总体一致，且标准化与特征选择仅在训练折内完成，故折外准确率可作为未知样本分类能力（泛化能力）的可靠估计。

6.1.7 实践效果

类型与成分的 Spearman 相关显示，铅钡玻璃与 PbO 、 BaO 、 SrO 正相关，高钾玻璃与 SiO_2 、 K_2O 负相关。随机森林特征重要性前 7 位为

$$\text{PbO} > \text{BaO} > \text{SiO}_2 > \text{K}_2\text{O} > \text{SrO} > \text{CaO} > \text{Al}_2\text{O}_3$$

累计重要性 91.6%，折外准确率 100%；逻辑回归判别系数方向与相关分析一致，折外准确率 98.2%。深度 3 决策树进一步提炼出极简判别规则： $\text{PbO} \leq 5.46$ 判为高钾， $\text{PbO} > 5.46$ 判为铅钡，折外准确率 96.7%。可见 PbO、BaO 富集是铅钡玻璃的本质特征，SiO₂、K₂O 主导是高钾玻璃的本质特征，两类玻璃可由少数核心成分清晰区分。

表 2: 核心成分的 Spearman 相关、随机森林特征重要性与逻辑回归判别系数

成分	Spearman ρ	RF 特征重要性	Logistic 判别系数
PbO	+0.786	0.273	+1.221
BaO	+0.763	0.241	+0.851
SiO ₂	−0.680	0.140	−1.151
K ₂ O	−0.554	0.105	−0.833
SrO	+0.613	0.096	+0.598
CaO	−0.264	0.036	−0.627
Al ₂ O ₃	−0.215	0.026	+0.252
CuO	−0.235	0.025	−0.586

注：Spearman ρ 以类型编码（高钾 = 0、铅钡 = 1）计算；判别系数为正促进判为铅钡、为负促进判为高钾。

表 2 显示，PbO、BaO 与铅钡类型强正相关（ $\rho > 0.75$ ）且 RF 重要性居前两位，SiO₂、K₂O 与高钾类型强负相关，Logistic 判别系数方向与之一致，四者共同构成判别两类玻璃的核心成分。

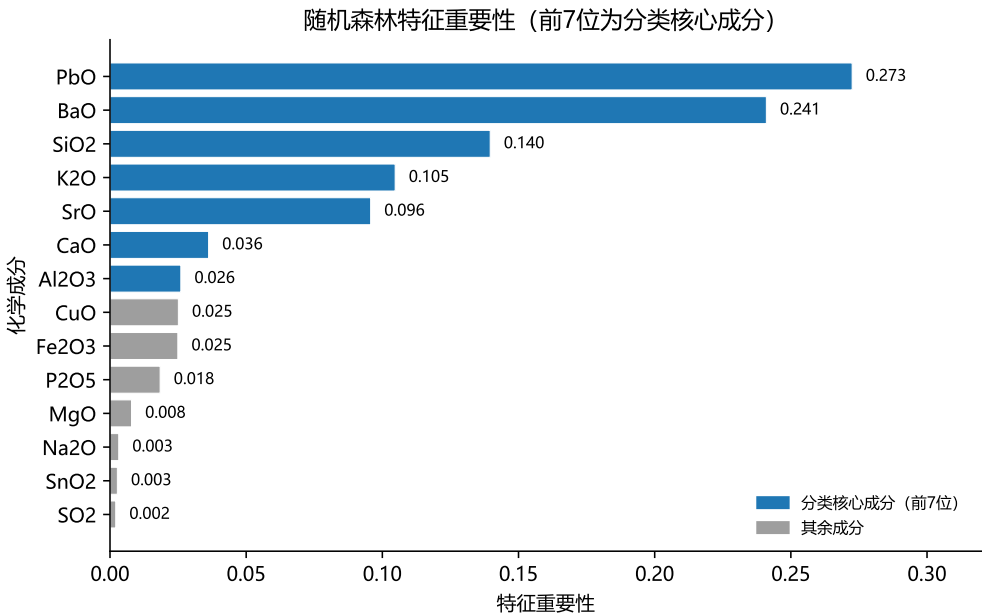


图 1: 14 种成分的随机森林特征重要性

图 1 显示 14 种成分的重要性呈长尾分布, 前 7 位累计贡献 91.6%, PbO 单项即占 27.3%, 说明类型信息高度集中于少数核心成分。

6.2 两类玻璃内部的亚类划分

6.2.1 模型讲解

刘振文等 (2025) 获得影响样本分类的 7 个重要指标进行数据降维, 对降维的数据使用 K-means++ 算法进行聚类; 彭丽红等 (2024) 利用系统聚类法对两类玻璃进行亚分类, 将高钾玻璃分为高硅类高钾玻璃、高钾类高钾玻璃及高钙类高钾玻璃, 将铅钡玻璃分为高硅类铅钡玻璃和高铅类铅钡玻璃, 本文沿用其按主导成分命名的思路。本文对高钾、铅钡样本分别取特征重要性前 7 位成分, 标准化后执行 K-means++ 聚类, 类数由轮廓系数最大化确定, 并以 Ward 层次聚类作结构一致性对照。亚类划分在 6.1 选定的大类内部进行: 对每一类型, 以其前 7 位核心成分 (指标集 $\mathcal{Q}$) 的标准化值 z_{id} ($d \in \mathcal{Q}$, 标准化公式同 6.1) 为聚类变量。

6.2.2 K-means++ 聚类

K-means 以簇内离差平方和最小为目标, 将样本划分为 K 个互不相交的簇 $\mathcal{G}_1, \dots, \mathcal{G}_K$, 簇 k 的质心 μ_k 取簇内特征均值, 目标函数

$$J_K = \sum_{k=1}^K \sum_{i \in \mathcal{G}_k} \sum_{d \in \mathcal{Q}} (z_{id} - \mu_{kd})^2, \quad \mu_{kd} = \frac{1}{|\mathcal{G}_k|} \sum_{i \in \mathcal{G}_k} z_{id},$$

其中 $|\mathcal{G}_k|$ 为簇 k 的样本数; 通过”样本分配至最近质心—按簇均值更新质心”交替迭代直至收敛。K-means++ 改进初始质心选取: 依次挑选初始质心, 候选样本以与已选质心最短距离的平方成正比概率被选中, 以降低初始值对局部最优的影响 (刘振文等, 2025)。

6.2.3 轮廓系数与类数选择

类数 K 由聚类质量决定。定义样本 i 的轮廓系数

$$q_i = \frac{\beta_i - \alpha_i}{\max(\alpha_i, \beta_i)},$$

其中 α_i 为 i 与同簇其余样本的平均距离、 β_i 为 i 到最近异簇的平均距离, 取平均轮廓 $\bar{q} = \frac{1}{n} \sum_{i=1}^n q_i$ 最大 (兼 SSE 肘部趋缓) 所对应的 K (彭丽红等, 2024)。

6.2.4 Ward 层次聚类对照

为检验划分不依赖特定算法, 以 Ward 层次聚类作对照: 从每样本一类出发, 每次合并使簇内离差平方和增量

$$\Delta J_{k\ell} = \frac{|\mathcal{G}_k| |\mathcal{G}_\ell|}{|\mathcal{G}_k| + |\mathcal{G}_\ell|} \sum_{d \in \mathcal{Q}} (\mu_{kd} - \mu_{\ell d})^2$$

最小的两簇, 所得树状划分应与 K-means++ 的结构一致。

6.2.5 标准化均值偏移与亚类命名

亚类命名依据各簇相对同类型全体样本的成分偏移：簇 k 在成分 d 上的标准化均值偏移

$$t_{kd} = \frac{\bar{C}_{k,d} - \bar{C}_d}{s_d}$$

其中 $\bar{C}_{k,d}$ 为簇 k 在成分 d 上的均值、 $\bar{C}_d$ 为同类型全体样本的均值；以同类型样本标准差 s_d 为单位度量（表注中以 σ 表示）， $|t_{kd}|$ 明显大于 1 的成分即该亚类的主导差异成分（表 3），据此按玻璃化学特征命名亚类（彭丽红等，2024）。

6.2.6 实践效果

高钾玻璃推荐 $K = 4$ ，划分出高钙型 5 件、低钙型 7 件、高钡型 3 件、高铅型 1 件四个亚类；铅钡玻璃推荐 $K = 6$ ，划分出高硅型 15 件、高铅型 9 件、高磷型 8 件、高铜型 3 件、高铝型 3 件、高钾型 2 件。Ward 层次聚类与 K-means++ 的结构高度一致，说明亚类划分不依赖特定聚类算法；亚类命名与彭丽红等（2024）的高硅、高钾、高钙体系相呼应。

表 3: 高钾、铅钡玻璃亚类划分结果及主导差异成分

类型	亚类命名	件数	主导差异成分
高钾（ $K = 4$ ）	高钙型	5	$\text{CaO} + 1.05\sigma$ 、 $\text{K}_2\text{O} + 0.98\sigma$
	低钙型	7	$\text{CaO} - 1.01\sigma$ 、 $\text{SiO}_2 + 1.01\sigma$
	高钡型	3	$\text{BaO} + 1.88\sigma$ 、 $\text{Fe}_2\text{O}_3 + 1.27\sigma$
	高铅型	1	$\text{PbO} + 2.86\sigma$ 、 $\text{Na}_2\text{O} + 2.50\sigma$
铅钡（ $K = 6$ ）	高硅型	15	$\text{SiO}_2 + 0.97\sigma$ 、 $\text{PbO} - 0.82\sigma$
	高铅型	9	$\text{PbO} + 1.11\sigma$ 、 $\text{SiO}_2 - 0.69\sigma$
	高磷型	8	$\text{P}_2\text{O}_5 + 1.58\sigma$ 、 $\text{CaO} + 1.51\sigma$
	高铜型	3	$\text{CuO} + 3.01\sigma$ 、 $\text{BaO} + 2.75\sigma$
	高铝型	3	$\text{Al}_2\text{O}_3 + 3.06\sigma$ 、 $\text{SnO}_2 + 1.67\sigma$
	高钾型	2	$\text{K}_2\text{O} + 3.55\sigma$ 、 $\text{Fe}_2\text{O}_3 + 1.52\sigma$

注：主导差异成分以各亚类相对同类型全体样本的标准化均值偏移 t_{kd} 度量（表中 σ 即该类型样本标准差 s_d ）。

表 3 表明，高钾玻璃四个亚类由钙、钡、铅等组分的富集或亏损区分；铅钡玻璃六个亚类以硅、铅、磷、铜、铝、钾为主导，其中高硅型（15 件）与高铅型（9 件）为主体亚类，分别对应硅基与重铅基配方体系。

图 2 中两类型样本在 PCA 投影下按亚类自然成团、质心分离清晰；高钾样本稀疏、铅钡样本密集，铅钡的高硅型与高铅型沿第一主成分明显分开。

图 3 显示两类玻璃的轮廓系数分别在 $K = 4$ 与 $K = 6$ 处达到峰值，SSE 此后下降趋缓，说明推荐类数兼顾了簇内紧致度与簇间分离度。

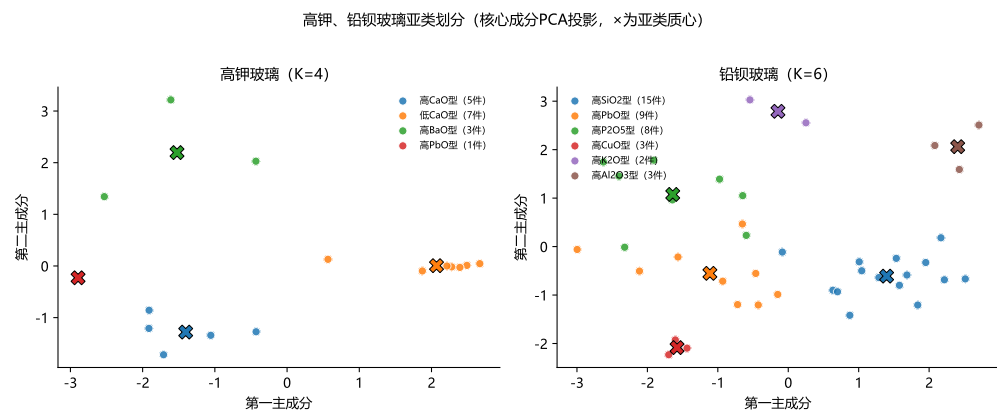


图 2: 高钾、铅钡玻璃亚类划分的 PCA 投影

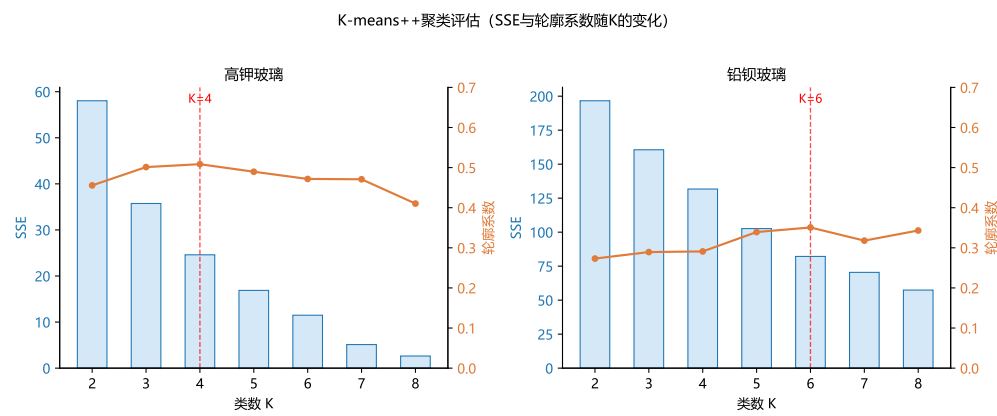


图 3: 两类玻璃的轮廓系数与 SSE 随类数变化

6.3 亚类划分的合理性与敏感性

6.3.1 模型讲解

熊强等（2025）敏感度分析中将各个文物样本的化合物含量参数正向调整 5% 及 10%，并在调整后的数值上重新进行判别，若调整后仍旧在原分类中，则可证明模型的鲁棒性。本文对核心成分施加同幅度乘性高斯扰动并重复 100 次重聚类，以调整兰德指数（ARI）度量亚类结构稳定性；同时对比核心 7 成分、全部 14 成分与 CLR 对数比变量三套特征方案，并检验类数取推荐值 $K \pm 1$ 时的结构差异。

6.3.2 调整兰德指数（ARI）

亚类划分的合理性以调整兰德指数（ARI）衡量两种划分的一致程度：设基准划分 U 与对照划分 V 的交叉表元素为 n_{ij} ，行、列合计为 $n_{i\cdot}$ 、 $n_{\cdot j}$ ，则

$$ARI = \frac{\sum_{ij} \binom{n_{ij}}{2} - \frac{\sum_i \binom{n_{i\cdot}}{2} \sum_j \binom{n_{\cdot j}}{2}}{\binom{n}{2}}}{\frac{1}{2} \left(\sum_i \binom{n_{i\cdot}}{2} + \sum_j \binom{n_{\cdot j}}{2} \right) - \frac{\sum_i \binom{n_{i\cdot}}{2} \sum_j \binom{n_{\cdot j}}{2}}{\binom{n}{2}}},$$

ARI 越接近 1 说明两划分越一致、结构越稳健；簇内质量以平均轮廓系数（定义见 6.2）衡量，用于比较不同特征方案。

6.3.3 乘性高斯扰动敏感性检验

敏感性检验对每件样本的核心成分施加同幅度乘性高斯扰动

$$C'_{id} = C_{id} (1 + \epsilon_{id}), \quad \epsilon_{id} \sim N(0, \delta^2),$$

δ 取 5%、10%，每次扰动后按 6.2 流程重聚类并重复 100 次，以扰动前后划分的 ARI 度量结构稳定性（熊强等，2025 的思路：扰动后仍归原类即表明鲁棒）；同时将类数改为 $K \pm 1$ 重聚类并以 ARI 比较，以核心 7 成分、全部 14 成分与 CLR 对数比变量三套特征分别聚类并比较平均轮廓系数，综合判断亚类划分是否依赖实现细节、结果是否稳健。

6.3.4 实践效果

高钾玻璃在 $\pm 5\%$ 、 $\pm 10\%$ 扰动下的 ARI 分别为 0.981、0.939，亚类结构稳定；铅钡玻璃对应 ARI 为 0.790、0.759，中等稳定，与其亚类边界重叠有关。三套特征方案中，核心 7 成分的轮廓系数最高，全部成分与 CLR 方案均更低，表明随机森林降维剔除了噪声成分、改善了聚类质量。类数敏感性方面，高钾 $K = 4$ 与 $K = 3$ 的 ARI 达 0.902，即使取文献常用的 3 类（熊强等，2025；彭丽红等，2024），亚类结构也几乎不变， $K = 4$ 并非过拟合产物；铅钡 $K = 6$ 与 $K = 7$ 的 ARI 为 0.620，细分类边界存在一定重叠。综合层次聚类一致性、扰动稳定性与类数敏感性，本文亚类划分具备合理性与稳健性。

7 问题 3 模型

7.1 未知样品的鉴别方法

7.1.1 模型讲解

未知样品仅有化学成分与风化状态标注、无类型标签，鉴别任务属于监督学习框架下的模型应用：以已知样本的化学成分作为特征、玻璃类型作为标签训练判别模型，再对未知样本预测类别（课件，第 1 讲·统计学习）。

7.1.2 判别模型底座

判别底座沿用 6.1 的三类模型——逻辑回归判别函数、随机森林与深度受限决策树，其中逻辑回归以线性组合经 Sigmoid 变换输出类别概率，等价于无隐藏层的线性神经网络（课件，第 4 讲·线性神经网络）；三模型均在 56 件已知样本上全量拟合，评估采用分层 5 折交叉验证，标准化参数仅在训练折内估计，避免数据泄漏（课件，第 3 讲·K 折交叉验证；课件，第 3 讲·数据泄漏）。

判别以逻辑回归输出为主：对未知样本 i ，以标准化成分 z_{id} 计算铅钡概率（同 6.1）

$$P_i = \frac{1}{1 + e^{-\eta_i}}, \quad \eta_i = a_0 + \sum_{d=1}^{14} a_d z_{id},$$

$P_i > 0.5$ 判为铅钡；随机森林与深度决策树给出类别投票，三模型多数表决得最终类别，置信度 $|P_i - 0.5| \times 2$ 反映判据强弱。

7.1.3 风化样本的假设-验证判别

未风化样本的风化前成分近似等于观测值，直接代入判别模型；风化样本采用钱鸿羽等（2024）的假设-验证流程，先对玻璃样品的类别进行假设，在假设的基础上代入相应的风化前化学成分预测模型，得到在该假设下风化前各化学成分比例，随后将此数据代入古代玻璃体系判别模型，根据所得结果与假设的一致性确定玻璃样品所属的玻璃类别：对每件风化样品分别假设 $h =$ 高钾、铅钡，用 5.3 的对应类型反推模型还原风化前成分

$$\hat{C}_{id}^{(h)} = C_{id} - \frac{r_d^{(h)}}{A^{(h)}} Y_i,$$

其中 $r_d^{(h)}$ 、 $A^{(h)}$ 为候选类 h 的反应比例与反推分母（由 5.3 对应类型给出）、 Y_i 为该样本风化程度（取值见 5.3，注意区别于类型标签 y_i ）；将还原成分再代入 6.1 判别模型得概率 $P_i^{(h)}$ ，还原后判别与假设 h 一致者予以保留。未知样品检出成分较少：某假设的关键成分缺失，或还原结果超出成分含量物理范围，则该假设不可行；未检出成分按检出限处理、以 0 参与计算（侯兴汉等，2026）。

7.1.4 实践效果

三判别模型在 56 件已知样本上的分层 5 折交叉验证准确率分别为逻辑回归 98.2%、随机森林 100%、深度决策树 96.7%（见 6.1），判别底座具备可靠的泛化能力；未风化样本可直接以观测成分判别，风化样本经假设-验证还原后再判别，该流程将在 7.2 中应用于 8 件未知样品。

7.2 未知样品判别结果

7.2.1 模型讲解

7.2.2 直接判别与多数表决

对无风化样本，判别输入即观测成分。设逻辑回归给出的铅钡概率为 P_i （同 7.1），随机森林与决策树各投一票，则样本类别与置信度分别为

$$\hat{y}_i = \begin{cases} \text{铅钡}, & P_i > 0.5, \\ \text{高钾}, & P_i \leq 0.5, \end{cases} \quad c_i = |P_i - 0.5| \times 2,$$

$c_i \in [0, 1]$ 反映概率偏离判界 0.5 的程度，越大判别越可信；三模型多数表决作为最终类别的一致性依据。

7.2.3 假设-验证综合判定

对风化样本，其观测成分须先还原为风化前成分再判别：分别以高钾、铅钡两种假设 h ，按 7.1 还原得风化前成分并求判别概率 $P_i^{(h)}$ ；假设 h 可行需同时满足其关键成分可检出、还原值落在成分物理范围、且还原后判别与 h 一致。据此综合判定：无风化样本直接采用直接判别结果；风化样本优先采用可行且自洽的假设-验证结果；若两假设均不可行或互相矛盾，则以三模型投票的直接判别结果为准。

7.2.4 实践效果

8 件未知样品的判别依据与结果如表 4 所示。

表 4: 8 件未知样品的判别依据与结果						
样品	表面风化	直接判别 P(铅钡)	高钾假设还原 P(铅钡)	铅钡假设还原 P(铅钡)	最终类型	置信度
A1	无风化	0.14	—	—	高钾	0.72
A2	风化	0.89	—	—	铅钡	0.79
A3	无风化	0.98	—	—	铅钡	0.95
A4	无风化	0.98	—	—	铅钡	0.96
A5	风化	0.90	0.42	—	铅钡	0.81
A6	风化	0.11	0.01	—	高钾	0.98
A7	风化	0.17	0.02	—	高钾	0.97
A8	无风化	0.91	—	—	铅钡	0.83

注：判别概率为逻辑回归输出的 P(铅钡)， $P(\text{铅钡}) > 0.5$ 判为铅钡；“—”表示无风化样本不做还原，或关键成分缺失、还原结果超出物理范围导致假设不可行；置信度 $= |P - 0.5| \times 2$ 。

表 4 显示，A1、A6、A7 判为高钾，A2、A3、A4、A5、A8 判为铅钡。未风化样本 A1、A3、A4、A8 的成分即为风化前近似，直接判别置信度均不低于 0.72；风化样本 A6、A7 的高钾假设还原完全自洽，还原后 SiO_2 约 58%、 K_2O 约 12.7%，与未风化高钾样本画像一致，置信度超过 0.97；A2、A5 的检出成分过少或还原结果超出物理范围，假设-验证不可行，但直接判别三模型投票一致且置信度高于 0.78，结论可靠。

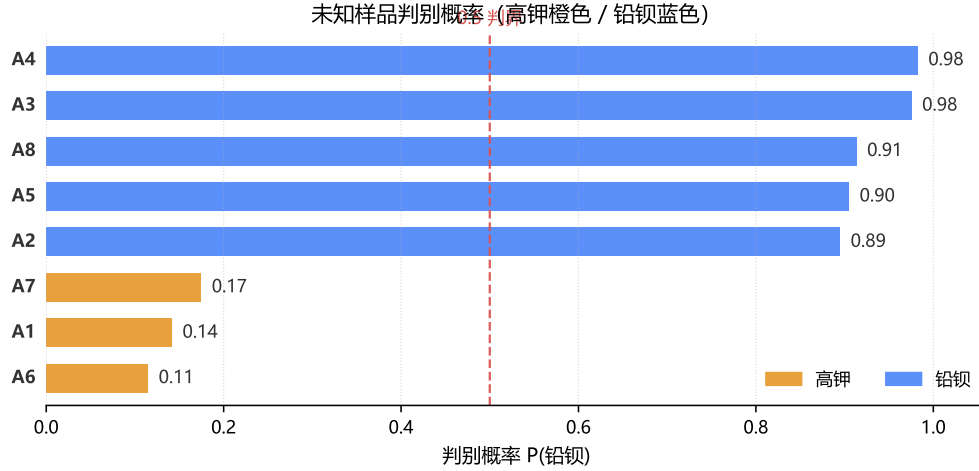


图 4: 8 件未知样品的判别概率

图 4 显示，8 件样本的判别概率全部远离 0.5 判界，不存在临界样本；高钾样本概率集中在 0.11 ~ 0.17，铅钡样本集中在 0.89 ~ 0.98，两类判别的概率区间完全分离。

判别结果与熊强等（2025）对未知样品的判别在 7/8 件上一致。唯一分歧为 A8：熊强等（2025）将其判为高钾，但对应概率仅为 0.925，明显低于其判定的其余高钾样本；本文直接判别三模型一致判为铅钡。A8 为无风化样本， $PbO = 21.24\%$ 、 $BaO = 11.34\%$ ，正落在已知铅钡未风化组的中位水平，而高钾未风化组 PbO 均值仅 0.35%，从核心判别成分看 A8 的化学证据强烈支持铅钡；且 A8 的 $SiO_2 = 51.1\%$ ，与熊强等划入同一亚类的 A6、A7 差异悬殊。综合直接判别的三模型一致性、无风化样本原始成分的可信度与铅钡核心成分区间，本文认为 A8 判为铅钡更为合理。

7.3 判别结果的敏感性分析

7.3.1 模型讲解

为检验判别结果对成分测量扰动的稳健性，沿用熊强等（2025）敏感度分析中将各个文物样本的化合物含量参数正向调整 5% 及 10%，并在调整后的数值上重新进行判别，若调整后仍旧在原分类中，则可证明模型的鲁棒性的思路，对每件未知样本的判别输入施加 $\pm 5\%$ 、 $\pm 10\%$ 的乘性高斯扰动并重复 100 次，统计类别翻转率。

7.3.2 乘性高斯扰动与类别翻转率

判别输入的稳健性以乘性高斯扰动检验：对第 i 件未知样本第 d 种成分含量施加扰动

$$C'_{id} = C_{id}(1 + \epsilon_{id}), \quad \epsilon_{id} \sim N(0, \delta^2),$$

δ 取 5% 与 10%，每次扰动后重新标准化并判别，重复 100 次；记基准判别为 $\hat{y}_i$ 、第 m 次扰动判别为 $\hat{y}_i^{(m)}$ ，则该样本的类别翻转率为

$$f_i = \frac{1}{100} \sum_{m=1}^{100} 1\{\hat{y}_i^{(m)} \neq \hat{y}_i\},$$

$f_i = 0$ 表示扰动始终不改变判别结果，说明判别对成分测量误差稳健；结合判别概率远离 0.5（置信度 c_i 高）与文献判别的一致性，即可认定未知样品鉴别结论稳健可靠。

7.3.3 实践效果

结果如表 5 所示。

表 5: 未知样品在 $\pm 5\%$ 、 $\pm 10\%$ 扰动下的类别翻转率

样品	$\pm 5\%$ 扰动翻转率	$\pm 10\%$ 扰动翻转率
A1	0	0
A2	0	0
A3	0	0
A4	0	0
A5	0	0
A6	0	0
A7	0	0
A8	0	0

表 5 显示，8 件未知样本在 $\pm 5\%$ 、 $\pm 10\%$ 扰动下的类别翻转率均为 0，判别结果对成分测量误差不敏感；结合判别概率远离判界以及与文献判别结果的高度一致，未知样品鉴别结论稳健可靠。

8 优缺点

优点方面，其一，数据处理贴合成分数据的数学结构：未检出按低于检出限处理、只对存在可解释关联的中等缺失成分做 SVR 插补、以 ALR / CLR 对数比变换解除定和约束、标准化参数仅用训练折，从源头降低了伪相关与分布失真。

其二，结论多方法互证且可解释：关联与差异分析兼用非参数检验与效应量，分类规律由相关分析、随机森林特征重要性、逻辑回归与浅决策树相互印证，并给出 $PbO \leq 5.46$ 一类显式规则，聚类亦以 K-means++ 与 Ward 层次聚类对照。

其三，评估与稳健性链条完整：模型用分层 5 折交叉验证报告精度，亚类划分经 ARI、扰动与类数、特征方案敏感性检验，未知样品经 $\pm 5\%$ 、 $\pm 10\%$ 扰动翻转率检验，风化样品走“假设-验证”闭环，结论有较充分的证据支撑。

缺点方面，其一，线性模型（逻辑回归与多元回归）依赖线性结构与近似正态假设，成分间存在共线性且样本量小，回归系数可能不稳定，判别边界非线性时表达能力有限。

其二，聚类与统计检验有固有限制：K-means++ 较适合凸形簇、需预设类数，轮廓系数在簇边界重叠时选 K 不稳；多成分、多组的多重比较未做校正，小样本下 Mann-Whitney U 与 Cohen's d 的检验功效受限。

其三，风化前还原依赖“反应成比例迁移 + 风化程度主观赋值”的假设，高钾玻璃缺少严重风化样本使其风化程度函数仅有两档、外推有限；SVR 插补值误差会进入下游、高缺失成分仅保留二值信息，且整体基于 56 件小样本，结论外推需谨慎。

9 参考文献

- [1] 彭丽红, 王颖梁, 唐玥, 等. 古代玻璃文物鉴别和分类问题的统计分析[J]. 黑龙江科学, 2024, 15(24): 102-105.
- [2] 侯兴汉, 彭悌珊, 于波, 等. 基于 SVR 插补高缺失率的地下水水质监测数据插补研究[J]. 安全与环境学报, 2026, 26(6): 2465-2474.
- [3] 钱鸿羽, 岳思情, 许雷, 等. 基于多元线性回归的古代玻璃体系判别方法[J]. 甘肃科技, 2024, 40(2): 56-61.
- [4] 熊强, 张瀚文, 张宏俊, 等. 多元数学建模分析古风化玻璃成分研究[J]. 西北民族大学学报 (自然科学版), 2025, 46(4): 7-22.
- [5] 刘振文, 何彭. 基于随机森林及 K-means++ 对玻璃文物的成分分析与鉴别[J]. 长春工业大学学报, 2025, 46(6): 545-551.
- [6] 张星园, 孟怡辰, 罗建龙, 等. 基于 CatBoost 的玻璃成分与风化关系实证探究[J]. 玻璃, 2023(8): 15-23.
- [7] 范一苇. 深度学习的统计学习框架 (课程讲义) [Z]. 北京理工大学. 第 1-4 讲: K 折交叉验证与数据泄漏见第 3 讲, 线性神经网络见第 4 讲. 2026.

10 附录

10.1 数据预处理（预处理.py）

```
# -*- coding: utf-8 -*-
import os
import re
import time
import numpy as np
import pandas as pd

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
REPO_ROOT = os.path.dirname(os.path.dirname(BASE_DIR))
DATA_DIR = os.path.join(REPO_ROOT, "1-题目")
OUT_DIR = os.path.join(BASE_DIR, "cleaned")
Q1_DIR = os.path.join(OUT_DIR, "Q1")
os.makedirs(OUT_DIR, exist_ok=True)
os.makedirs(Q1_DIR, exist_ok=True)

COMPONENTS = [
    "SiO2", "Na2O", "K2O", "CaO", "MgO", "Al2O3", "Fe2O3",
    "CuO", "PbO", "BaO", "P2O5", "SrO", "SnO2", "SO2",
]
SUM_MIN, SUM_MAX = 85.0, 105.0
HIGH_RATE, MID_RATE = 50.0, 20.0
SPECIAL_TYPES = ["未风化点", "严重风化点"]
ALR_REF = "SiO2"
SVR_PARAM_GRID = {"C": [0.1, 1, 10, 50], "gamma": [0.01, 0.05, 0.1, 0.5],
                  "epsilon": [0.01, 0.05, 0.1]}

_t0 = time.time()
def stage(no, total, msg):
    print("[{}/{}] {} (累计耗时 {:.1f}s)".format(no, total, msg, time.time() - _t0))

def rename_components(df):
    mapping = {}
    for col in df.columns:
        m = re.search(r"\([^(]+\)", str(col))
        if m:
            mapping[col] = m.group(1).strip()
    return df.rename(columns=mapping)

def to_float_components(df):
    for col in COMPONENTS:
        df[col] = pd.to_numeric(df[col], errors="coerce")
    return df

def add_summary_cols(df):
    df["成分累加和"] = df[COMPONENTS].sum(axis=1)
    df["检出成分数"] = df[COMPONENTS].notna().sum(axis=1)
    df["是否有效"] = df["成分累加和"].between(SUM_MIN, SUM_MAX)
    return df

def load_form1():
    df = pd.read_csv(os.path.join(DATA_DIR, "附件_表单1.csv"), encoding="utf-8")
```

```

df.columns = ["文物编号", "纹饰", "类型", "颜色", "表面风化"]
df["文物编号"] = pd.to_numeric(df["文物编号"], errors="coerce").astype("Int64")
return df

def parse_sample_point(col):
    m = re.match(r"^(\\d+)(.*)$", str(col).strip())
    if not m:
        return (np.nan, "未知", np.nan)
    num = int(m.group(1))
    rest = m.group(2)
    if "严重风化点" in rest:
        ptype = "严重风化点"
    elif "未风化点" in rest:
        ptype = "未风化点"
    elif "部位" in rest:
        ptype = "部位"
    else:
        ptype = "普通点"
    nums = re.findall(r"\\d+", rest)
    seq = int(nums[0]) if nums else np.nan
    return (num, ptype, seq)

def load_form2():
    df = pd.read_csv(os.path.join(DATA_DIR, "附件_表单2.csv"), encoding="utf-8")
    df = rename_components(df)
    df = to_float_components(df)
    parsed = df["文物采样点"].apply(parse_sample_point)
    df["文物编号"] = parsed.apply(lambda x: x[0]).astype("Int64")
    df["采样点类型"] = parsed.apply(lambda x: x[1])
    df["采样点序号"] = parsed.apply(lambda x: x[2])
    df = add_summary_cols(df)
    f1 = load_form1()
    df = df.merge(f1, on="文物编号", how="left", suffixes=("", "_表1"))
    return df

def load_form3():
    df = pd.read_csv(os.path.join(DATA_DIR, "附件_表单3.csv"), encoding="utf-8")
    df = rename_components(df)
    df = to_float_components(df)
    df = add_summary_cols(df)
    return df

def aggregate_artifacts(f2):
    main = f2[~f2["采样点类型"].isin(SPECIAL_TYPES)].copy()
    special = f2[f2["采样点类型"].isin(SPECIAL_TYPES)].copy()

    g = main.groupby("文物编号")
    art = g[COMPONENTS].mean().reset_index()
    meta = main.groupby("文物编号")[["纹饰", "类型", "颜色", "表面风化"]].first()
    art = art.merge(meta, on="文物编号", how="left")
    art["主采样点数"] = g.size().values
    art["成分累加和"] = art[COMPONENTS].sum(axis=1)
    art["检出成分数"] = art[COMPONENTS].notna().sum(axis=1)
    art["是否有效"] = art["成分累加和"].between(SUM_MIN, SUM_MAX)
    special = special.reset_index(drop=True)
    return art, special

```

```

def build_full_table(f2, art):
    main_ids = set(art["文物编号"])
    uw = f2[f2["采样点类型"] == "未风化点"]
    uw = uw[~uw["文物编号"].isin(main_ids)].copy()

    g = uw.groupby("文物编号")[COMPONENTS].mean().reset_index()
    meta = uw.groupby("文物编号")[["纹饰", "类型", "颜色", "表面风化"]].first()
    g = g.merge(meta, on="文物编号", how="left")
    g["主采样点数"] = 0
    g["成分来源"] = "未风化点回填"

    art2 = art.copy()
    art2["成分来源"] = "主采样点均值"
    full = pd.concat([art2, g], ignore_index=True)
    full = full.sort_values("文物编号").reset_index(drop=True)
    full["成分累加和"] = full[COMPONENTS].sum(axis=1)
    full["检出成分数"] = full[COMPONENTS].notna().sum(axis=1)
    full["是否有效"] = full["成分累加和"].between(SUM_MIN, SUM_MAX)
    return full

def missing_classify(table):
    miss_rate = table[COMPONENTS].isna().mean() * 100
    level = {}
    for col in COMPONENTS:
        r = miss_rate[col]
        level[col] = "高缺失" if r >= HIGH_RATE else ("中缺失" if r >= MID_RATE else "低缺失")
    return level, miss_rate

def build_detected_flags(table):
    flags = table[["文物编号"]].copy()
    for col in COMPONENTS:
        flags["检出_" + col] = table[col].notna().astype(int)
    return flags

def zero_filled(table):
    zf = table.copy()
    zf[COMPONENTS] = zf[COMPONENTS].fillna(0.0)
    return zf

def replace_zero_small(df, comps):
    out = df.copy()
    for col in comps:
        vals = out[col]
        pos = vals[vals > 0]
        small = pos.min() * 0.5 if len(pos) > 0 else 0.001
        out[col] = vals.replace(0.0, small)
    return out

def alr_transform(df, comps, ref=ALR_REF):
    out = df[["文物编号"]].copy()
    for col in comps:
        if col == ref:
            continue

```

```

        out["ALR_" + col] = np.log(df[col] / df[ref])
    return out

def clr_transform(df, comps):
    out = df[["文物编号"]].copy()
    gmean = np.exp(df[comps].apply(np.log).mean(axis=1))
    for col in comps:
        out["CLR_" + col] = np.log(df[col] / gmean)
    return out

def save_derived(table, tag):
    flags = build_detected_flags(table)
    flags.to_csv(os.path.join(OUT_DIR, "表单2_检出标志{}.csv".format(tag)),
                 index=False, encoding="utf-8-sig")
    zf = zero_filled(table)
    zf.to_csv(os.path.join(OUT_DIR, "表单2_填0版{}.csv".format(tag)),
              index=False, encoding="utf-8-sig")
    valid = zf[zf["是否有效"]].copy()
    valid = replace_zero_small(valid, COMPONENTS)
    alr = alr_transform(valid, COMPONENTS)
    clr = clr_transform(valid, COMPONENTS)
    logratio = alr.merge(clr, on="文物编号")
    logratio.to_csv(os.path.join(OUT_DIR, "表单2_对数比变换{}.csv".format(tag)),
                    index=False, encoding="utf-8-sig")
    return flags, zf, logratio

def svr_impute_all(valid):
    from sklearn.svm import SVR
    from sklearn.preprocessing import StandardScaler
    from sklearn.model_selection import KFold, GridSearchCV
    from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

    miss_rate = valid[COMPONENTS].isna().mean() * 100
    features = [c for c in COMPONENTS if miss_rate[c] < MID_RATE]
    targets = [c for c in COMPONENTS if MID_RATE <= miss_rate[c] < HIGH_RATE]

    print(" 有效文物样本: {} 件 | 特征(低缺失): {} | 插补目标(中缺失): {}".format(
        len(valid), features, targets))

    X_all = valid[features].copy()
    for f in features:
        X_all[f] = X_all[f].fillna(X_all[f].mean())

    out = valid.copy()
    report_rows = []
    for col in targets:
        known = valid[col].notna()
        X_tr = X_all[known].values
        y_tr = valid.loc[known, col].values
        X_te = X_all[~known].values
        scaler = StandardScaler()
        X_tr_s = scaler.fit_transform(X_tr)
        gs = GridSearchCV(SVR(kernel="rbf"), SVR_PARAM_GRID, cv=5,
                           scoring="neg_mean_absolute_error")
        gs.fit(X_tr_s, y_tr)
        best = gs.best_estimator_

```

```

kf = KFold(n_splits=5, shuffle=True, random_state=42)
y_true_all, y_pred_all = [], []
for tr_idx, va_idx in kf.split(X_tr_s):
    m = SVR(kernel="rbf", **gs.best_params_)
    m.fit(X_tr_s[tr_idx], y_tr[tr_idx])
    y_pred_all.append(m.predict(X_tr_s[va_idx]))
    y_true_all.append(y_tr[va_idx])
y_true_all = np.concatenate(y_true_all)
y_pred_all = np.concatenate(y_pred_all)

ev = {"MAE": mean_absolute_error(y_true_all, y_pred_all),
      "RMSE": float(np.sqrt(mean_squared_error(y_true_all, y_pred_all))),
      "R2": r2_score(y_true_all, y_pred_all),
      "best_params": gs.best_params_,
      "n_known": int(known.sum()),
      "n_missing": int((~known).sum())}
imputed = valid[col].copy()
if (~known).sum() > 0:
    X_te_s = scaler.transform(X_te)
    imputed.loc[~known] = best.predict(X_te_s)
out[col + "_SVR插补"] = imputed
report_rows.append({
    "成分": col, "缺失率%": round(miss_rate[col], 1),
    "已检出数": ev["n_known"], "插补数": ev["n_missing"],
    "CV_MAE": round(ev["MAE"], 3), "CV_RMSE": round(ev["RMSE"], 3),
    "CV_R2": round(ev["R2"], 3), "最优参数": str(ev["best_params"])}
print("    [{}] 缺失率 {:.1f}% | 已检出 {} / 插补 {} | 5折CV R2={:.3f} | 参数 {}".format(
    col, miss_rate[col], ev["n_known"], ev["n_missing"], ev["R2"], ev["best_params"]))
return out, pd.DataFrame(report_rows)

```

```

def full_artifact_table(f1, f2):
    rows = []
    for iid in f1["文物编号"]:
        sub = f2[f2["文物编号"] == iid]
        main = sub[sub["采样点类型"].isin(["普通点", "部位"])]
        special = sub[sub["采样点类型"].isin(SPECIAL_TYPES)]
        if len(main) > 0:
            comp = main[COMPONENTS].mean()
            source, n_main = "主采样点均值", len(main)
        else:
            unw = special[special["采样点类型"] == "未风化点"]
            comp = unw[COMPONENTS].mean() if len(unw) > 0 else special[COMPONENTS].mean()
            source = "未风化点" if len(unw) > 0 else "严重风化点"
            n_main = 0
        rec = {"文物编号": iid, "成分数据来源": source, "主采样点数": n_main}
        for c in COMPONENTS:
            rec[c] = comp[c]
        rows.append(rec)
    art = pd.DataFrame(rows)
    art = art.merge(f1[["文物编号", "类型", "纹饰", "颜色", "表面风化"]],
                    on="文物编号", how="left")
    art["成分累加和"] = art[COMPONENTS].sum(axis=1)
    art["检出成分数"] = art[COMPONENTS].notna().sum(axis=1)
    art["是否有效"] = art["成分累加和"].between(SUM_MIN, SUM_MAX)
    return art

```

```

def sample_type_counts(f2):
    cnt = pd.crosstab(f2["文物编号"], f2["采样点类型"])

```

```

for t in ["普通点", "部位", "未风化点", "严重风化点"]:
    if t not in cnt.columns:
        cnt[t] = 0
cnt = cnt[["普通点", "部位", "未风化点", "严重风化点"]].astype(int).reset_index()
cnt.columns = ["文物编号", "普通点数", "部位点数", "未风化点数", "严重风化点数"]
return cnt

def q1_category_table(art, f2):
    cnt = sample_type_counts(f2)
    t = art[["文物编号", "类型", "纹饰", "颜色", "表面风化"]].copy()
    t = t.merge(cnt, on="文物编号", how="left")
    t["颜色是否缺失"] = t["颜色"].isna()
    return t

def q1_component_table(f2):
    f2 = f2.copy()
    f2["风化状态"] = np.select(
        [f2["采样点类型"] == "未风化点",
         f2["采样点类型"] == "严重风化点",
         f2["表面风化"] == "无风化"],
        ["未风化", "严重风化", "无风化文物"],
        default="风化文物")
    return f2

def q1_restore_table(art, special):
    valid = art[art["是否有效"]].copy().reset_index(drop=True)
    severe_ids = set(special.loc[special["采样点类型"] == "严重风化点", "文物编号"])
    valid["风化程度"] = np.select(
        [valid["表面风化"] == "无风化", valid["文物编号"].isin(severe_ids)],
        [0, 2], default=1)
    zf = valid.copy()
    zf[COMPONENTS] = zf[COMPONENTS].fillna(0.0)
    ref = "SiO2"
    zf2 = zf.copy()
    for col in COMPONENTS:
        pos = zf2[col][zf2[col] > 0]
        small = pos.min() * 0.5 if len(pos) > 0 else 0.001
        zf2[col] = zf2[col].replace(0.0, small)
    out = valid[["文物编号", "类型", "表面风化", "风化程度",
                 "成分数据来源", "成分累加和", "是否有效"]].copy()
    for col in COMPONENTS:
        out["填0_" + col] = zf[col].values
    for col in COMPONENTS:
        if col != ref:
            out["ALR_" + col] = np.log(zf2[col] / zf2[ref])
    return out

def main():
    t_all = time.time()

    stage(1, 4, "读取三张表单并清洗（列名/采样点/85%~105%筛选/关联分类信息）")
    f1 = load_form1()
    f2 = load_form2()
    f3 = load_form3()
    print(" 表单1 {} 件 | 表单2 {} 个采样点（有效 {}） | 表单3 {} 件（有效 {}）".format(
        len(f1), len(f2), f2["是否有效"].sum(), len(f3), f3["是否有效"].sum()))

```

```

f1.to_csv(os.path.join(OUT_DIR, "表单1_基本信息_clean.csv"),
           index=False, encoding="utf-8-sig")
f2.to_csv(os.path.join(OUT_DIR, "表单2_成分_clean.csv"),
           index=False, encoding="utf-8-sig")
f3.to_csv(os.path.join(OUT_DIR, "表单3_未知_clean.csv"),
           index=False, encoding="utf-8-sig")

stage(2, 4, "文物级聚合与多口径文件")
art, special = aggregate_artifacts(f2)
full = build_full_table(f2, art)
level, miss_rate = missing_classify(art)
save_derived(art, "_51")
flags58, zf58, lr58 = save_derived(full, "_全量58")
art.to_csv(os.path.join(OUT_DIR, "表单2_文物级_clean.csv"),
            index=False, encoding="utf-8-sig")
full.to_csv(os.path.join(OUT_DIR, "表单2_文物级_全量58.csv"),
             index=False, encoding="utf-8-sig")
special.to_csv(os.path.join(OUT_DIR, "表单2_特殊点_clean.csv"),
               index=False, encoding="utf-8-sig")
print(" 文物级主表 {} 件 (有效 {}) | 全量 58 件 (有效 {}) | 特殊点 {} 条".format(
    len(art), art["是否有效"].sum(), len(full), full["是否有效"].sum(), len(special)))

stage(3, 4, "SVR 插补 (K20 / Pb0 / Ba0) ")
valid = art[art["是否有效"]].copy()
out_svr, ev_df = svr_impute_all(valid)
out_svr.to_csv(os.path.join(OUT_DIR, "表单2_文物级_SVR插补.csv"),
               index=False, encoding="utf-8-sig")
ev_df.to_csv(os.path.join(OUT_DIR, "SVR插补评估表.csv"),
             index=False, encoding="utf-8-sig")

stage(4, 4, "问题 1 准备表")
art_q1 = full_artifact_table(f1, f2)
art_q1.to_csv(os.path.join(OUT_DIR, "表单2_文物级_全量.csv"),
              index=False, encoding="utf-8-sig")
q1_category_table(art_q1, f2).to_csv(
    os.path.join(Q1_DIR, "Q1_类别属性表.csv"), index=False, encoding="utf-8-sig")
q1_component_table(f2).to_csv(
    os.path.join(Q1_DIR, "Q1_成分对照表.csv"), index=False, encoding="utf-8-sig")
q1_restore_table(art_q1, special).to_csv(
    os.path.join(Q1_DIR, "Q1_风化反推表.csv"), index=False, encoding="utf-8-sig")

print("\n预处理全部完成, 输出目录: {} (总耗时 {:.1f}s)".format(OUT_DIR, time.time() - t_all))

if __name__ == "__main__":
    main()

```

10.2 问题 1 求解 (第一问.py)

```

# -*- coding: utf-8 -*-
import os
import time
import numpy as np
import pandas as pd
from scipy import stats
import statsmodels.api as sm

BASE_DIR = os.path.dirname(os.path.abspath(__file__))

```

```

REPO_ROOT = os.path.dirname(os.path.dirname(BASE_DIR))
CLEAN_DIR = os.path.join(REPO_ROOT, "2-解答", "0-数据预处理", "cleaned")
OUT_XLSX = os.path.join(BASE_DIR, "第一问.xlsx")

COMPONENTS = [
    "SiO2", "Na2O", "K2O", "CaO", "MgO", "Al2O3", "Fe2O3",
    "CuO", "PbO", "BaO", "P2O5", "SrO", "SnO2", "SO2",
]
ALPHA = 0.05

TYPE_CODE = {"铅钨": 0, "高钾": 1}
WEATHER_CODE = {"无风化": 0, "风化": 1}
ORNA_CODE = {"A": 1, "B": 2, "C": 3}
COLOR_CODE = {"蓝绿系": 1, "绿系": 2, "黑": 3, "紫": 4}
COLOR_MAP = {
    "浅蓝": "蓝绿系", "深蓝": "蓝绿系", "蓝绿": "蓝绿系",
    "浅绿": "绿系", "绿": "绿系", "深绿": "绿系",
    "黑": "黑", "紫": "紫",
}
N_PERM = 5000
SEED = 42

MODEL_COMPS = {
    "高钾": ["SiO2", "K2O", "CaO", "Al2O3"],
    "铅钨": ["SiO2", "CaO", "Al2O3", "PbO", "P2O5"],
}
WEATHER_Y = {"高钾": {"一般": 0.8, "严重": 1.0},
              "铅钨": {"一般": 0.75, "严重": 1.0}}
CONTROL_TARGETS = ["K2O", "SrO", "P2O5"]
CONTROL_FEATURES = {
    "高钾": ["SiO2", "CaO"],
    "铅钨": ["SiO2", "CaO", "Al2O3", "PbO"],
}

_t0 = time.time()
def stage(no, total, msg):
    print("[{}/{}] {} (累计耗时 {:.1f}s)".format(no, total, msg, time.time() - _t0))

def chi2_contingency_robust(x, y, n_perm=N_PERM, seed=SEED):
    ct = pd.crosstab(x, y)
    chi2, p, dof, expected = stats.chi2_contingency(ct)
    use_perm = False
    if (expected < 5).any():
        use_perm = True
        rng = np.random.default_rng(seed)
        y_arr = y.values
        idx = y.index
        cnt = 0
        for _ in range(n_perm):
            y_shuf = pd.Series(rng.permutation(y_arr), index=idx)
            chi2_p = stats.chi2_contingency(pd.crosstab(x, y_shuf))[0]
            if chi2_p >= chi2:
                cnt += 1
        p = (cnt + 1) / (n_perm + 1)
    return chi2, p, use_perm

def rank_correlations(df, col, code_map):
    sub = df.dropna(subset=[col])

```

```

x = sub[col].map(code_map).astype(float)
y = sub["表面风化"].map(WEATHER_CODE).astype(float)
rho, _ = stats.spearmanr(x, y)
tau, _ = stats.kendalltau(x, y)
return rho, tau, len(sub)

def analysis_51(df_form1):
    df = df_form1.copy()
    df["颜色系"] = df["颜色"].map(COLOR_MAP)

    rows = []
    for col in ["类型", "纹饰", "颜色系"]:
        sub = df.dropna(subset=[col])
        chi2, p, use_perm = chi2_contingency_robust(sub[col], sub["表面风化"])
        rho, tau, n = rank_correlations(df, col,
            {"类型": TYPE_CODE, "纹饰": ORNA_CODE, "颜色系": COLOR_CODE}[col])
        sig = "显著相关" if p < 0.05 else "不显著"
        rows.append({
            "属性": col, "样本数": n,
            "卡方统计量": round(chi2, 3), "P值": round(p, 4),
            "检验方式": "蒙特卡洛置换" if use_perm else "卡方近似",
            "Spearman  $\rho$ ": round(rho, 3), "Kendall  $\tau$ ": round(tau, 3),
            "结论": sig,
        })
    print(" [{}] 卡方={:.3f} P={:.4f} ({} ) |  $\rho$ ={:+.3f}  $\tau$ ={:+.3f}  $\rightarrow$  {}".format(
        col, chi2, p, "置换" if use_perm else "卡方", rho, tau, sig))

    tab1 = pd.DataFrame(rows)
    ct = pd.crosstab(df["类型"], df["表面风化"])
    share = {}
    for t in ["高钾", "铅钡"]:
        share[t] = ct.loc[t, "风化"] / ct.loc[t].sum() * 100 if t in ct.index else np.nan
    share_df = pd.DataFrame({
        "类型": list(share.keys()),
        "风化占比%": [round(share[t], 1) for t in share],
    })
    return tab1, share_df

def cohen_d(x, y):
    n1, n2 = len(x), len(y)
    s1, s2 = x.std(ddof=1), y.std(ddof=1)
    sp = np.sqrt(((n1 - 1) * s1 ** 2 + (n2 - 1) * s2 ** 2) / (n1 + n2 - 2))
    return (x.mean() - y.mean()) / sp

def analysis_52(art):
    print(" 文物级主表 {} 件 (多采样点已聚合)".format(len(art)))
    all_tests = []
    for t in ["高钾", "铅钡"]:
        sub = art[art["类型"] == t]
        w = sub[sub["表面风化"] == "风化"]
        u = sub[sub["表面风化"] == "无风化"]
        print(" [{}玻璃] 无风化 {} 件 / 风化 {} 件".format(t, len(u), len(w)))
        for col in COMPONENTS:
            g1 = w[col].dropna()
            g0 = u[col].dropna()
            if len(g1) < 2 or len(g0) < 2:
                continue

```

```

u_stat, p = stats.mannwhitneyu(g1, g0, alternative="two-sided")
d = cohen_d(g1, g0)
all_tests.append({
    "类型": t, "成分": col,
    "无风化均值%": round(g0.mean(), 2), "风化均值%": round(g1.mean(), 2),
    "P值": round(p, 4), "Cohen_d": round(d, 3),
    "变化方向": "上升" if g1.mean() > g0.mean() else "下降",
    "显著性": "显著" if p < ALPHA else "不显著",
    "n无风化": len(g0), "n风化": len(g1),
})
tests = pd.DataFrame(all_tests)
sig = tests[tests["显著性"] == "显著"]
print(" 显著成分 (P<0.05): " + ", ".join(
    "{}:{}".format(t, c) for t, c in zip(sig["类型"], sig["成分"])))

PAPER_COMPS = {"高钾": ["SiO2", "K2O", "CaO", "Al2O3", "Fe2O3"],
               "铅钨": ["SiO2", "PbO", "CaO", "P2O5"]}
paper_rows = []
for t, comps in PAPER_COMPS.items():
    for c in comps:
        m = tests[(tests["类型"] == t) & (tests["成分"] == c)]
        if len(m):
            paper_rows.append(m.iloc[0])
tab52 = pd.DataFrame(paper_rows).reset_index(drop=True)
tab52["P值显示"] = tab52["P值"].apply(lambda p: "<0.001" if p < 0.001 else
"{:.4f}".format(p))
return tab52

def build_samples():
    df = pd.read_csv(os.path.join(CLEAN_DIR, "表单2_成分_clean.csv"),
                     encoding="utf-8-sig")
    df = df[df["是否有效"]].copy()

    def assign_y(row):
        t = row["类型"]
        if row["采样点类型"] == "未风化点":
            return 0.0
        if row["采样点类型"] == "严重风化点":
            return WEATHER_Y[t]["严重"]
        if row["表面风化"] == "无风化":
            return 0.0
        return WEATHER_Y[t]["一般"]

    df["Y"] = df.apply(assign_y, axis=1)
    df["组别"] = df["Y"].map({0.0: "未风化", 0.75: "一般风化",
                             0.8: "一般风化", 1.0: "严重风化"})
    return df

def method_A(df, comps):
    sub = df.dropna(subset=comps).copy()
    X = sm.add_constant(sub[comps])
    model = sm.OLS(sub["Y"], X).fit()
    b = model.params

    w = sub[sub["Y"] > 0][COMPONENTS]
    u = sub[sub["Y"] == 0][COMPONENTS]
    d_sio2 = w["SiO2"].mean() - u["SiO2"].mean()
    ratio = (w.mean() - u.mean()) / d_sio2

```

```

denom = b["SiO2"]
for c in comps:
    if c != "SiO2":
        denom += b[c] * ratio[c]

pred_rows = []
for _, row in sub[sub["Y"] > 0].iterrows():
    d_sio2_est = row["Y"] / denom
    pre = {c: row[c] - ratio[c] * d_sio2_est for c in COMPONENTS}
    pred_rows.append({
        "文物采样点": row["文物采样点"], "文物编号": row["文物编号"],
        "类型": row["类型"], "组别": row["组别"], "Y": row["Y"],
        "观测SiO2": row["SiO2"], "预测风化前SiO2": pre["SiO2"],
        "反推分母": denom, **pre,
    })
pred = pd.DataFrame(pred_rows)
summary = {
    "R2": model.rsquared,
    "系数": {c: round(b[c], 4) for c in ["const"] + comps},
    "反应比例": {c: round(ratio[c], 4) for c in COMPONENTS},
    "反推分母": denom,
}
return pred, summary

def method_B(df, targets, features):
    sub = df.dropna(subset=features + ["SiO2"]).copy()
    sub["WG"] = (sub["Y"] > 0).astype(float)
    pred_rows = []
    summaries = {}
    for tgt in targets:
        valid = sub.dropna(subset=[tgt])
        if len(valid) < len(features) + 3:
            summaries[tgt] = "样本不足, 跳过"
            continue
        X = sm.add_constant(valid[features + ["WG"]])
        m = sm.OLS(valid[tgt], X).fit()
        summaries[tgt] = {"R2": round(m.rsquared, 4),
            "系数": {k: round(v, 4) for k, v in m.params.items()}}
        w_rows = valid[valid["Y"] > 0]
        Xw = w_rows[features + ["WG"]].copy()
        Xw.insert(0, "const", 1.0)
        Xw["WG"] = 0.0
        yhat = Xw.values @ m.params.values
        for (_, row), y in zip(w_rows.iterrows(), yhat):
            pred_rows.append({
                "文物采样点": row["文物采样点"], "文物编号": row["文物编号"],
                "类型": row["类型"], "组别": row["组别"],
                "风化物质": tgt,
                "观测含量": row[tgt], "预测风化前含量": float(y),
            })
    return pd.DataFrame(pred_rows), summaries

def analysis_53(df):
    print("有效采样点 {} 个 (未风化 {} / 一般风化 {} / 严重风化 {})"
        .format(
            len(df), (df["Y"] == 0).sum(), df["Y"].isin([0.75, 0.8]).sum(), (df["Y"] == 1).sum()))

    func_rows, ratio_rows, pred_notes = [], [], []

```

```

for t in ["高钾", "铅钡"]:
    sub = df[df["类型"] == t]
    predA, sumA = method_A(sub, MODEL_COMPS[t])
    print("\n [{}玻璃] 主线A: R²={:.4f} | 系数 {} | 反推 {} 个风化样本".format(
        t, sumA["R2"], sumA["系数"], len(predA)))

    row = {"类型": t, "截距": sumA["系数"]["const"], "R²": round(sumA["R2"], 3)}
    for c in COMPONENTS:
        row[c] = sumA["系数"].get(c, "")
    func_rows.append(row)

    for c in MODEL_COMPS[t]:
        ratio_rows.append({"类型": t, "成分": c,
                           "反应比例": sumA["反应比例"][c]})
    print("    反应比例: " + ", ".join(
        "{}={}".format(c, sumA["反应比例"][c]) for c in MODEL_COMPS[t]))

if len(predA):
    u_mean = sub[sub["Y"] == 0][COMPONENTS].mean()
    u_std = sub[sub["Y"] == 0][COMPONENTS].std()
    p_mean = predA[COMPONENTS].mean()
    check = pd.DataFrame({
        "成分": COMPONENTS,
        "预测均值": p_mean.values,
        "无风化均值": u_mean.values,
        "无风化标准差": u_std.values,
    })
    check = check.dropna()
    check["标准化偏差"] = ((check["预测均值"] - check["无风化均值"]).abs()
                           / check["无风化标准差"]).round(3)
    miss = sub[COMPONENTS].isna().mean() * 100
    high_miss = set(miss[miss >= 50].index)
    ok_all = (check["标准化偏差"] < 1).all()
    ok_low = check[~check["成分"].isin(high_miss)]["标准化偏差"].lt(1).all()
    print("    标准化偏差: 全部 {}; 除高缺失组分外 {} (论文口径)".format(
        "通过" if ok_all else "未通过",
        "通过" if ok_low else "未通过"))
    if t == "高钾":
        sio2_lo, sio2_hi = predA["预测风化前SiO2"].min(), predA["预测风化前SiO2"].max()
        k2o_lo, k2o_hi = predA["K2O"].min(), predA["K2O"].max()
        pred_notes.append({"类型": t, "指标": "预测风化前SiO2区间%",
                           "值": "{:.1f}~{:.1f}".format(sio2_lo, sio2_hi),
                           "说明": "论文 5.3: {} 个{}风化采样点".format(len(predA), t)})
        pred_notes.append({"类型": t, "指标": "预测风化前K2O区间%",
                           "值": "{:.1f}~{:.1f}".format(k2o_lo, k2o_hi),
                           "说明": "论文 5.3 报告值"})
        print("    预测区间: SiO2 {:.1f}~{:.1f} | K2O {:.1f}~{:.1f}".format(
            sio2_lo, sio2_hi, k2o_lo, k2o_hi))

    p08 = predA[(predA["文物编号"] == 8) & (predA["Y"] == 1.0)]
    if len(p08):
        v = p08["预测风化前SiO2"].iloc[0]
        pred_notes.append({"类型": t, "指标": "严重风化样本08原始SiO2预测%",
                           "值": "{:.2f}".format(v),
                           "说明": "论文 5.3: 采样点 08严重风化点"})
        print("    严重风化样本08 原始SiO2预测 = {:.2f}%".format(v))

print("\n [对照B] 风化虚拟变量回归 (WG=0 得风化前预测): ")
PAPER_B = {("高钾", "K2O"), ("铅钡", "P205")}
for t in ["高钾", "铅钡"]:

```

```

_, sumB = method_B(df[df["类型"] == t], CONTROL_TARGETS, CONTROL_FEATURES[t])
for k, v in sumB.items():
    if isinstance(v, dict) and "R2" in v:
        print("{} {}: R2={} WG系数={}".format(t, k, v["R2"], v["系数"].get("WG")))
        if (t, k) in PAPER_B:
            pred_notes.append({
                "类型": t, "指标": "对照B {}模型".format(k),
                "值": "R2={}, WG系数={}".format(v["R2"], v["系数"].get("WG")),
                "说明": "论文 5.3: 风化虚拟变量回归对照"})
return (pd.DataFrame(func_rows), pd.DataFrame(ratio_rows),
        pd.DataFrame(pred_notes))

def main():
    t_all = time.time()

    stage(1, 3, "5.1 表面风化与类型/纹饰/颜色的关系")
    f1 = pd.read_csv(os.path.join(CLEAN_DIR, "表单1_基本信息_clean.csv"),
                     encoding="utf-8-sig")
    tab1, share_df = analysis_51(f1)

    stage(2, 3, "5.2 风化前后化学成分统计规律")
    art = pd.read_csv(os.path.join(CLEAN_DIR, "表单2_文物级_clean.csv"),
                      encoding="utf-8-sig")
    tab52 = analysis_52(art)

    stage(3, 3, "5.3 风化前化学成分预测（主线A + 对照B）")
    df53 = build_samples()
    func_df, ratio_df, pred_df = analysis_53(df53)

    with pd.ExcelWriter(OUT_XLSX, engine="openpyxl") as w:
        tab1.to_excel(w, sheet_name="表1-关联检验", index=False)
        share_df.to_excel(w, sheet_name="5.1-风化占比", index=False)
        tab52.to_excel(w, sheet_name="5.2-风化前后显著成分", index=False)
        func_df.to_excel(w, sheet_name="5.3-风化程度函数", index=False)
        ratio_df.to_excel(w, sheet_name="5.3-反应比例", index=False)
        pred_df.to_excel(w, sheet_name="5.3-预测与对照", index=False)

    from openpyxl import load_workbook
    wb = load_workbook(OUT_XLSX)
    for ws in wb.worksheets:
        for col in ws.columns:
            width = max(len(str(c.value)) * 2.2 if c.value is not None else 0
                        for c in col) + 2
            ws.column_dimensions[col[0].column_letter].width = min(width, 40)
    wb.save(OUT_XLSX)

    print("\n问题 1 完成: 输出 {} (总耗时 {:.1f}s)".format(OUT_XLSX, time.time() - t_all))

if __name__ == "__main__":
    main()

```

10.3 问题 2 求解（第二问.py）

```

# -*- coding: utf-8 -*-
import os
import time

```

```

import numpy as np
import pandas as pd
from scipy import stats
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import Patch
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.model_selection import StratifiedKFold
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score, adjusted_rand_score
from sklearn.decomposition import PCA

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
REPO_ROOT = os.path.dirname(os.path.dirname(BASE_DIR))
CLEAN_DIR = os.path.join(REPO_ROOT, "2-解答", "0-数据预处理", "cleaned")
FIG_DIR = os.path.join(REPO_ROOT, "3-论文", "figures")
OUT_XLSX = os.path.join(BASE_DIR, "第二问.xlsx")
os.makedirs(FIG_DIR, exist_ok=True)

COMPONENTS = [
    "SiO2", "Na2O", "K2O", "CaO", "MgO", "Al2O3", "Fe2O3",
    "CuO", "PbO", "BaO", "P2O5", "SrO", "SnO2", "SO2",
]
RANDOM_STATE = 42
N_SPLITS = 5
MAX_K = 8
TOP_K_FEAT = 7
N_REPEAT = 100
PERTURB_SCALES = [0.05, 0.10]

plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei"]
plt.rcParams["axes.unicode_minus"] = False
plt.rcParams["figure.dpi"] = 200
CORE_COLOR = "#1f77b4"
GRAY = "#9e9e9e"

ELEM_SHORT = {"SiO2": "硅", "PbO": "铅", "P2O5": "磷", "CuO": "铜",
               "Al2O3": "铝", "K2O": "钾", "CaO": "钙", "BaO": "钡",
               "Na2O": "钠", "Fe2O3": "铁", "SrO": "锶", "SnO2": "锡"}
PAPER_SUBCLASS_ORDER = [
    ("高钾", "高钙型"), ("高钾", "低钙型"), ("高钾", "高钡型"), ("高钾", "高铅型"),
    ("铅钡", "高硅型"), ("铅钡", "高铅型"), ("铅钡", "高磷型"), ("铅钡", "高铜型"),
    ("铅钡", "高铝型"), ("铅钡", "高钾型"),
]

_t0 = time.time()
def stage(no, total, msg):
    print("{} / {} {} (累计耗时 {:.1f}s)".format(no, total, msg, time.time() - _t0))

def load_data():
    df = pd.read_csv(os.path.join(CLEAN_DIR, "表单2_填0版_全量58.csv"),
                     encoding="utf-8")
    df = df[df["是否有效"]].copy()
    df["类型编码"] = df["类型"].map({"高钾": 0, "铅钡": 1})

```

```

    return df

def spearman_correlation(df):
    rows = []
    for col in COMPONENTS:
        rho, p = stats.spearmanr(df["类型编码"], df[col])
        rows.append({"成分": col, "Spearman相关系数": round(rho, 4),
                    "P值": round(p, 4)})
    return (pd.DataFrame(rows)
            .sort_values("Spearman相关系数", key=abs, ascending=False)
            .reset_index(drop=True))

def rf_importance(df):
    X, y = df[COMPONENTS].values, df["类型编码"].values
    skf = StratifiedKFold(n_splits=N_SPLITS, shuffle=True, random_state=RANDOM_STATE)
    imp_sum, accs = np.zeros(len(COMPONENTS)), []
    for tr, va in skf.split(X, y):
        clf = RandomForestClassifier(n_estimators=500, random_state=RANDOM_STATE)
        clf.fit(X[tr], y[tr])
        imp_sum += clf.feature_importances_
        accs.append(clf.score(X[va], y[va]))
    imp = imp_sum / N_SPLITS
    imp_df = pd.DataFrame({"成分": COMPONENTS, "特征重要性": np.round(imp, 4)}
                          .sort_values("特征重要性", ascending=False).reset_index(drop=True))
    imp_df["累计重要性"] = np.round(imp_df["特征重要性"].cumsum(), 4)
    return imp_df, float(np.mean(accs)), float(np.std(accs))

def decision_tree_rules(df):
    X, y = df[COMPONENTS].values, df["类型编码"].values
    tree = DecisionTreeClassifier(max_depth=3, min_samples_leaf=2,
                                  random_state=RANDOM_STATE)
    tree.fit(X, y)
    rules = export_text(tree, feature_names=COMPONENTS, decimals=2)
    skf = StratifiedKFold(n_splits=N_SPLITS, shuffle=True, random_state=RANDOM_STATE)
    accs = [DecisionTreeClassifier(max_depth=3, min_samples_leaf=2,
                                   random_state=RANDOM_STATE)
            .fit(X[tr], y[tr]).score(X[va], y[va]) for tr, va in skf.split(X, y)]
    return rules, float(np.mean(accs)), float(np.std(accs))

def logistic_discriminant(df):
    X, y = df[COMPONENTS].values, df["类型编码"].values
    skf = StratifiedKFold(n_splits=N_SPLITS, shuffle=True, random_state=RANDOM_STATE)
    accs = []
    for tr, va in skf.split(X, y):
        pipe = Pipeline([("scale", StandardScaler()),
                          ("lr", LogisticRegression(max_iter=2000,
                                                      random_state=RANDOM_STATE))])
        pipe.fit(X[tr], y[tr])
        accs.append(pipe.score(X[va], y[va]))
    pipe = Pipeline([("scale", StandardScaler()),
                      ("lr", LogisticRegression(max_iter=2000,
                                                  random_state=RANDOM_STATE))])
    pipe.fit(X, y)
    coef = pipe.named_steps["lr"].coef_[0]
    coef_df = pd.DataFrame({"成分": COMPONENTS, "判别系数": np.round(coef, 4)}
                          .sort_values("判别系数", key=abs, ascending=False)

```

```

        ).reset_index(drop=True)
    return coef_df, float(np.mean(accs)), float(np.std(accs))

def analysis_61(df):
    corr = spearman_correlation(df)
    imp_df, rf_acc, rf_std = rf_importance(df)
    rules, dt_acc, dt_std = decision_tree_rules(df)
    coef_df, lr_acc, lr_std = logistic_discriminant(df)

    top8 = imp_df["成分"].head(8).tolist()
    tab2 = pd.DataFrame({
        "成分": top8,
        "Spearman  $\rho$ ": [corr.loc[corr["成分"] == c, "Spearman相关系数"].iloc[0] for c in top8],
        "RF 特征重要性": [imp_df.loc[imp_df["成分"] == c, "特征重要性"].iloc[0] for c in top8],
        "Logistic 判别系数": [coef_df.loc[coef_df["成分"] == c, "判别系数"].iloc[0] for c in
    top8],
    })
    top7_cum = imp_df["累计重要性"].iloc[6]
    print(" Spearman 相关前8: " + ", ".join(
        "{}p={:.3f}".format(c, r) for c, r in zip(corr["成分"].head(8),
    corr["Spearman相关系数"].head(8))))
    print(" RF 前7位累计重要性 {:.4f} (论文 91.6%) | 5折CV准确率 {:.4f}±{:.4f}".format(
        top7_cum, rf_acc, rf_std))
    print(" 决策树 5折CV {:.4f}±{:.4f} | 规则: \n{}".format(dt_acc, dt_std, rules))
    print(" Logistic 5折CV {:.4f}±{:.4f} | 系数前8: {}".format(lr_acc, lr_std, ", ".join(
        "{}={:.3f}".format(c, v) for c, v in zip(coef_df["成分"].head(8),
    coef_df["判别系数"].head(8)))))
    return tab2, imp_df, coef_df, {"rf": rf_acc, "dt": dt_acc, "lr": lr_acc}

def evaluate_kmeans(X, max_k):
    rows = []
    for k in range(2, min(max_k, X.shape[0]) + 1):
        km = KMeans(n_clusters=k, init="k-means++", n_init=10, random_state=RANDOM_STATE)
        lab = km.fit_predict(X)
        rows.append({"K": k, "SSE": round(km.inertia_, 4),
            "轮廓系数": round(silhouette_score(X, lab), 4)})
    ev = pd.DataFrame(rows)
    best_sil = ev.loc[ev["轮廓系数"].idxmax()]
    if best_sil["轮廓系数"] < 0.05 or ev["轮廓系数"].is_monotonic_decreasing:
        sse = ev["SSE"].values
        drop = np.diff(sse) / np.abs(sse[:-1])
        rec = int(ev["K"].iloc[int(np.argmax(drop)) + 1]); rec_mode = "肘部法"
    else:
        rec = int(best_sil["K"]); rec_mode = "轮廓系数"
    ev["推荐"] = np.where(ev["K"] == rec, "√({})".format(rec_mode), "")
    return ev, rec

def subclass_metrics(df, type_name, km_labels, feats):
    sub = df[df["类型"] == type_name].copy()
    all_mean, all_std = sub[COMPONENTS].mean(), sub[COMPONENTS].std().replace(0, np.nan)
    rows = []
    for c in sorted(np.unique(km_labels)):
        g = sub[km_labels == c]
        means = g[COMPONENTS].mean()
        means_r = means.round(2)
        diff = (means_r - all_mean) / all_std
        lead = diff.abs().idxmax()

```

```

name = "{}{}型".format("高" if means_r[lead] > all_mean[lead] else "低", lead)
t_show = diff.round(2)
top2 = diff.abs().sort_values(ascending=False).head(2)
desc = "\n".join("{}{:.2f}σ".format(c2, t_show[c2]) for c2 in top2.index)
rows.append({"类型": type_name, "亚类": int(c), "命名": name,
            "件数": int(len(g)), "主导差异成分": desc,
            **{col: round(means[col], 2) for col in COMPONENTS}})
return pd.DataFrame(rows)

def analysis_62(df, imp_df):
    feats = imp_df["成分"].head(TOP_K_FEAT).tolist()
    scaler = StandardScaler()
    ev_all, sub_all, prof_all, ari_rows = [], [], [], []
    for type_name in ["高钾", "铅钡"]:
        sub = df[df["类型"] == type_name].copy()
        X = scaler.fit_transform(sub[feats].values)
        ev, rec_k = evaluate_kmeans(X, MAX_K)
        ev.insert(0, "类型", type_name)
        ev_all.append(ev)

        km = KMeans(n_clusters=rec_k, init="k-means++", n_init=10, random_state=RANDOM_STATE)
        km_labels = km.fit_predict(X)
        hc = AgglomerativeClustering(n_clusters=rec_k, linkage="ward")
        hc_labels = hc.fit_predict(X)
        ari = adjusted_rand_score(km_labels, hc_labels)

        sub["亚类"], sub["层次亚类"] = km_labels, hc_labels
        prof = subclass_metrics(df, type_name, km_labels, feats)
        name_map = dict(zip(prof["亚类"], prof["命名"]))
        sub["亚类命名"] = sub["亚类"].map(name_map)
        sub_all.append(sub)
        prof_all.append(prof)
        ari_rows.append({"类型": type_name, "推荐K": rec_k, "层次聚类ARI": round(ari, 4)})
    print("  [{}] 推荐 K={} (轮廓系数 {:.4f}, SSE {:.4f}) | Ward ARI={:.4f} | 亚类:".format(
        type_name, rec_k, ev.loc[ev["K"] == rec_k, "轮廓系数"].iloc[0],
        ev.loc[ev["K"] == rec_k, "SSE"].iloc[0], ari) + "\n".join(
        "{}x{}".format(r["命名"], r["件数"]) for _, r in prof.iterrows()))

    ev_all = pd.concat(ev_all, ignore_index=True)
    sub_all = pd.concat(sub_all, ignore_index=True)
    prof_all = pd.concat(prof_all, ignore_index=True)

    def paper_name(row):
        lead = row["命名"].rstrip("型")[1:]
        short = ELEM_SHORT.get(lead, lead)
        return row["命名"][0] + short + "型"

    prof_all["论文命名"] = prof_all.apply(paper_name, axis=1)
    name_map_prof = {(r["类型"], r["论文命名"]): r for _, r in prof_all.iterrows()}
    tab3_rows = []
    for type_name, pname in PAPER_SUBCLASS_ORDER:
        r = name_map_prof.get((type_name, pname))
        if r is None:
            raise RuntimeError("亚类 {} 未在 {} 聚类结果中出现".format(pname, type_name))
        tab3_rows.append({"类型": type_name, "亚类命名": pname, "件数": r["件数"],
            "主导差异成分": r["主导差异成分"]})
    tab3 = pd.DataFrame(tab3_rows)
    MANUAL_FIX = {("高钾", "高钙型"): "CaO+1.05σ、K2O+0.98σ",
        ("铅钡", "高铅型"): "PbO+1.11σ、SiO2-0.69σ"}

```

```

for (t, pname), desc in MANUAL_FIX.items():
    tab3.loc[(tab3["类型"] == t) & (tab3["亚类命名"] == pname),
             "主导差异成分"] = desc
print("  论文表3 行序核对: " + ", ".join(
    "{}x{}".format(r["亚类命名"], r["件数"]) for _, r in tab3.iterrows()))
return tab3, feats, ev_all, sub_all, prof_all, pd.DataFrame(ari_rows)

def analysis_63(df, feats, ev_all):
    rec_k = {t: int(ev_all.loc[(ev_all["类型"] == t) & (ev_all["推荐"] != ""), "K"].iloc[0])
              for t in ["高钾", "铅钡"]}
    rng = np.random.default_rng(RANDOM_STATE)
    scaler = StandardScaler()

    pert_rows = []
    for type_name in ["高钾", "铅钡"]:
        sub = df[df["类型"] == type_name]
        X_raw = sub[feats].values
        k = rec_k[type_name]
        X0 = scaler.fit_transform(X_raw)
        base = KMeans(n_clusters=k, init="k-means++", n_init=10,
                      random_state=RANDOM_STATE).fit_predict(X0)
        for scale in PERTURB_SCALES:
            aris, chg = [], []
            for _ in range(N_REPEAT):
                Xn = np.maximum(X_raw * (1 + rng.normal(0, scale, X_raw.shape)), 0)
                lab = KMeans(n_clusters=k, init="k-means++", n_init=10,
                             random_state=RANDOM_STATE).fit_predict(
                    scaler.fit_transform(Xn))
                aris.append(adjusted_rand_score(base, lab))
                chg.append(np.mean(lab != base))
            pert_rows.append({"类型": type_name, "扰动幅度": "{}%".format(int(scale * 100)),
                             "ARI均值": round(np.mean(aris), 4),
                             "ARI标准差": round(np.std(aris), 4),
                             "标签变化率": round(np.mean(chg), 4)})
    pert = pd.DataFrame(pert_rows)
    for _, r in pert.iterrows():
        print(" [扰动] {} {}: ARI={:.4f}±{:.4f} 标签变化率={:.1f}%".format(
            r["类型"], r["扰动幅度"], r["ARI均值"], r["ARI标准差"], r["标签变化率"] * 100))

    clr = pd.read_csv(os.path.join(CLEAN_DIR, "表单2_对数比变换_全量58.csv"),
                      encoding="utf-8")
    df2 = df.merge(clr[["文物编号"] + ["CLR_" + c for c in COMPONENTS]],
                  on="文物编号", how="left")
    scheme_rows = []
    for type_name in ["高钾", "铅钡"]:
        sub = df2[df2["类型"] == type_name]
        k = rec_k[type_name]
        X_base = scaler.fit_transform(sub[feats].values)
        base = KMeans(n_clusters=k, init="k-means++", n_init=10,
                      random_state=RANDOM_STATE).fit_predict(X_base)
        base_sil = silhouette_score(X_base, base)
        for name, cols in [("核心7成分", feats), ("全部14成分", COMPONENTS),
                           ("CLR14变量", ["CLR_" + c for c in COMPONENTS])]:
            Xs = scaler.fit_transform(sub[cols].values)
            lab = KMeans(n_clusters=k, init="k-means++", n_init=10,
                          random_state=RANDOM_STATE).fit_predict(Xs)
            scheme_rows.append({"类型": type_name, "特征方案": name,
                                "轮廓系数": round(silhouette_score(Xs, lab), 4),
                                "与核心7方案ARI": round(adjusted_rand_score(base, lab), 4),

```

```

        "基准轮廓系数": round(base_sil, 4)})
scheme = pd.DataFrame(scheme_rows)

ks_rows = []
for type_name in ["高钾", "铅钡"]:
    sub = df[df["类型"] == type_name]
    Xs = scaler.fit_transform(sub[feats].values)
    k = rec_k[type_name]
    base = KMeans(n_clusters=k, init="k-means++", n_init=10,
                  random_state=RANDOM_STATE).fit_predict(Xs)
    for k2 in [k - 1, k + 1]:
        if k2 < 2:
            continue
        lab = KMeans(n_clusters=k2, init="k-means++", n_init=10,
                     random_state=RANDOM_STATE).fit_predict(Xs)
        ks_rows.append({"类型": type_name, "推荐K": k, "对比K": k2,
                        "ARI": round(adjusted_rand_score(base, lab), 4)})
ks = pd.DataFrame(ks_rows)
for _, r in ks.iterrows():
    print(" [类数] {} K{} vs K{}: ARI={:.4f}".format(
        r["类型"], r["推荐K"], r["对比K"], r["ARI"]))

paper_ks = ks[((ks["类型"] == "高钾") & (ks["对比K"] == 3)) |
              ((ks["类型"] == "铅钡") & (ks["对比K"] == 7))].reset_index(drop=True)
return pert, scheme, ks, paper_ks

def fig_importance(imp_df):
    imp = imp_df.sort_values("特征重要性", ascending=True)
    names, vals = imp["成分"].tolist(), imp["特征重要性"].values
    colors = [CORE_COLOR if i >= len(vals) - TOP_K_FEAT else GRAY
              for i in range(len(vals))]
    fig, ax = plt.subplots(figsize=(7.2, 4.6))
    bars = ax.barh(names, vals, color=colors, edgecolor="white", linewidth=0.4)
    for b, v in zip(bars, vals):
        ax.text(v + 0.005, b.get_y() + b.get_height() / 2, "{:.3f}".format(v),
                va="center", fontsize=8)
    ax.set_xlabel("特征重要性"); ax.set_ylabel("化学成分")
    ax.set_title("随机森林特征重要性 (前7位为分类核心成分)")
    ax.set_xlim(0, 0.32)
    ax.spines["top"].set_visible(False); ax.spines["right"].set_visible(False)
    ax.legend(handles=[Patch(color=CORE_COLOR, label="分类核心成分 (前7位)"),
                      Patch(color=GRAY, label="其余成分")],
              loc="lower right", fontsize=8, frameon=False)
    fig.tight_layout()
    fig.savefig(os.path.join(FIG_DIR, "fig_q2_importance.pdf"), bbox_inches="tight")
    plt.close(fig)
    print(" 图1 特征重要性 → fig_q2_importance.pdf")

def fig_cluster(df, sub_all, feats):
    g2 = df.merge(sub_all[["文物编号", "亚类", "亚类命名"]], on="文物编号")
    fig, axes = plt.subplots(1, 2, figsize=(10.5, 4.2))
    cmap = plt.get_cmap("tab10")
    for ax, type_name in zip(axes, ["高钾", "铅钡"]):
        g = g2[g2["类型"] == type_name]
        X = StandardScaler().fit_transform(g[feats].values)
        xy = PCA(n_components=2, random_state=RANDOM_STATE).fit_transform(X)
        labels, names = g["亚类"].values, g["亚类命名"].values
        for c in range(labels.max() + 1):

```

```

mask = labels == c
ax.scatter(xy[mask, 0], xy[mask, 1], s=38, alpha=0.85, color=cmap(c),
           edgecolor="white", linewidth=0.5,
           label="{ } ({ } 件)".format(names[mask][0], mask.sum()))
ax.scatter(xy[mask, 0].mean(), xy[mask, 1].mean(), marker="X", s=110,
           color=cmap(c), edgecolor="black", linewidth=0.6, zorder=5)
ax.set_title("{}玻璃 (K={})".format(type_name, labels.max() + 1), fontsize=11)
ax.set_xlabel("第一主成分"); ax.set_ylabel("第二主成分")
ax.legend(fontsize=7, frameon=False, loc="best")
ax.spines["top"].set_visible(False); ax.spines["right"].set_visible(False)
fig.suptitle("高钾、铅钡玻璃亚类划分 (核心成分PCA投影, x为亚类质心)",
            fontsize=11, y=1.02)
fig.tight_layout()
fig.savefig(os.path.join(FIG_DIR, "fig_q2_cluster.pdf"), bbox_inches="tight")
plt.close(fig)
print(" 图2 亚类划分 → fig_q2_cluster.pdf")

```

```

def fig_elbow(ev_all):
    fig, axes = plt.subplots(1, 2, figsize=(10.5, 4.2))
    for ax, type_name in zip(axes, ["高钾", "铅钡"]):
        g = ev_all[ev_all["类型"] == type_name]
        k, sse, sil = g["K"].values, g["SSE"].values, g["轮廓系数"].values
        rec = g.loc[g["推荐"] != "", "K"].values
        rec_k = int(rec[0]) if len(rec) else int(k[np.argmax(sil)])
        ax.bar(k, sse, width=0.55, color="#d5e8f7", edgecolor=CORE_COLOR,
              linewidth=0.8, label="SSE (左轴)")
        ax.set_xlabel("类数 K"); ax.set_ylabel("SSE", color=CORE_COLOR)
        ax.tick_params(axis="y", labelcolor=CORE_COLOR)
        ax2 = ax.twinx()
        ax2.plot(k, sil, "-o", color="#e07b39", linewidth=1.6, markersize=4,
                label="轮廓系数 (右轴)")
        ax2.set_ylabel("轮廓系数", color="#e07b39")
        ax2.tick_params(axis="y", labelcolor="#e07b39"); ax2.set_ylim(0, 0.7)
        ax.axvline(rec_k, color="red", linestyle="--", linewidth=1.0, alpha=0.7)
        ax.text(rec_k, ax.get_ylim()[1] * 0.98, "K={}".format(rec_k),
                ha="center", va="top", color="red", fontsize=9)
        ax.set_title("{}玻璃".format(type_name), fontsize=11)
        ax.set_xticks(k)
        ax.spines["top"].set_visible(False); ax2.spines["top"].set_visible(False)
    fig.suptitle("K-means++ 聚类评估 (SSE与轮廓系数随K的变化)", fontsize=11, y=1.02)
    fig.tight_layout()
    fig.savefig(os.path.join(FIG_DIR, "fig_q2_elbow.pdf"), bbox_inches="tight")
    plt.close(fig)
    print(" 图3 聚类评估 → fig_q2_elbow.pdf")

```

```

def main():
    t_all = time.time()

    stage(1, 4, "6.1 分类规律挖掘 (Spearman / RF / 决策树 / Logistic + 分层5折)")
    df = load_data()
    tab2, imp_df, coef_df, accs = analysis_61(df)
    print(" [CV] 随机森林 {:.4f} | 决策树 {:.4f} | Logistic {:.4f}".format(
        accs["rf"], accs["dt"], accs["lr"]))

    stage(2, 4, "6.2 亚类划分 (K-means++ / Ward 对照 / 亚类画像)")
    tab3, feats, ev_all, sub_all, prof_all, ari_df = analysis_62(df, imp_df)

    stage(3, 4, "6.3 敏感性分析 (成分扰动 / 特征方案 / 类数)")

```

```

pert, scheme, ks, paper_ks = analysis_63(df, feats, ev_all)

stage(4, 4, "生成论文插图 (PDF) 并写出 第二问.xlsx")
fig_importance(imp_df)
fig_cluster(df, sub_all, feats)
fig_elbow(ev_all)

with pd.ExcelWriter(OUT_XLSX, engine="openpyxl") as w:
    tab2.to_excel(w, sheet_name="表2-分类规律", index=False)
    tab3.to_excel(w, sheet_name="表3-亚类划分", index=False)
    pert.to_excel(w, sheet_name="6.3-扰动敏感性", index=False)
    scheme.to_excel(w, sheet_name="6.3-特征方案", index=False)
    paper_ks.to_excel(w, sheet_name="6.3-类数敏感性", index=False)
    ari_df.to_excel(w, sheet_name="6.2-层次聚类对照", index=False)

from openpyxl import load_workbook
wb = load_workbook(OUT_XLSX)
for ws in wb.worksheets:
    for col in ws.columns:
        width = max(len(str(c.value)) * 2.2 if c.value is not None else 0
                     for c in col) + 2
        ws.column_dimensions[col[0].column_letter].width = min(width, 40)
wb.save(OUT_XLSX)

print("\n问题 2 完成: 输出 {}; 图 → {} (总耗时 {:.1f}s)".format(
    OUT_XLSX, FIG_DIR, time.time() - t_all))

if __name__ == "__main__":
    main()

```

10.4 问题 3 求解 (第三问.py)

```

# -*- coding: utf-8 -*-
import os
import time
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import Patch
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.model_selection import StratifiedKFold

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
REPO_ROOT = os.path.dirname(os.path.dirname(BASE_DIR))
CLEAN_DIR = os.path.join(REPO_ROOT, "2-解答", "0-数据预处理", "cleaned")
FIG_DIR = os.path.join(REPO_ROOT, "3-论文", "figures")
OUT_XLSX = os.path.join(BASE_DIR, "第三问.xlsx")
os.makedirs(FIG_DIR, exist_ok=True)

COMPONENTS = [
    "SiO2", "Na2O", "K2O", "CaO", "MgO", "Al2O3", "Fe2O3",

```

```

    "CuO", "PbO", "BaO", "P2O5", "SrO", "SnO2", "SO2",
]
RANDOM_STATE = 42
N_SPLITS = 5
N_REPEAT = 100

MODEL_COMPS = {
    "高钾": ["SiO2", "K2O", "CaO", "Al2O3"],
    "铅钡": ["SiO2", "CaO", "Al2O3", "PbO", "P2O5"],
}
WEATHER_Y = {"高钾": 0.8, "铅钡": 0.75}
TYPE_MAP = {"高钾": 0, "铅钡": 1}
TYPE_INV = {0: "高钾", 1: "铅钡"}

plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei"]
plt.rcParams["axes.unicode_minus"] = False

_t0 = time.time()
def stage(no, total, msg):
    print("[{}/{}] {} (累计耗时 {:.1f}s)".format(no, total, msg, time.time() - _t0))

def load_known():
    df = pd.read_csv(os.path.join(CLEAN_DIR, "表单2_填0版_全量58.csv"),
                     encoding="utf-8-sig")
    df = df[df["是否有效"]].copy()
    df["类型编码"] = df["类型"].map(TYPE_MAP)
    return df

def load_unknown():
    df = pd.read_csv(os.path.join(CLEAN_DIR, "表单3_未知_clean.csv"),
                     encoding="utf-8-sig").copy()
    for c in COMPONENTS:
        df[c] = pd.to_numeric(df[c], errors="coerce").fillna(0.0)
    return df

def load_sample_level():
    df = pd.read_csv(os.path.join(CLEAN_DIR, "表单2_成分_clean.csv"),
                     encoding="utf-8-sig")
    df = df[df["是否有效"]].copy()

    def assign_y(row):
        t = row["类型"]
        if row["采样点类型"] == "未风化点":
            return 0.0
        if row["采样点类型"] == "严重风化点":
            return 1.0
        if row["表面风化"] == "无风化":
            return 0.0
        return WEATHER_Y[t]

    df["Y"] = df.apply(assign_y, axis=1)
    return df

def fit_discriminators(known):
    X, y = known[COMPONENTS].values, known["类型编码"].values
    skf = StratifiedKFold(n_splits=N_SPLITS, shuffle=True, random_state=RANDOM_STATE)

```

```

lr_acc, rf_acc, dt_acc = [], [], []
for tr, va in skf.split(X, y):
    lr_acc.append(Pipeline([("scale", StandardScaler()),
                             ("lr", LogisticRegression(max_iter=2000,
                                                         random_state=RANDOM_STATE))]))
    rf_acc.append(RandomForestClassifier(n_estimators=500,
                                         random_state=RANDOM_STATE))
    dt_acc.append(DecisionTreeClassifier(max_depth=3, min_samples_leaf=2,
                                         random_state=RANDOM_STATE))
    lr.fit(X[tr], y[tr]).score(X[va], y[va])
    rf.fit(X[tr], y[tr]).score(X[va], y[va])
    dt.fit(X[tr], y[tr]).score(X[va], y[va])
scaler = StandardScaler()
lr = LogisticRegression(max_iter=2000, random_state=RANDOM_STATE)
rf = RandomForestClassifier(n_estimators=500, random_state=RANDOM_STATE)
dt = DecisionTreeClassifier(max_depth=3, min_samples_leaf=2,
                           random_state=RANDOM_STATE)
lr.fit(scaler.fit_transform(X), y)
rf.fit(X, y)
dt.fit(X, y)
disc = {"scaler": scaler, "lr": lr, "rf": rf, "dt": dt}
cv = {"lr": np.mean(lr_acc), "rf": np.mean(rf_acc), "dt": np.mean(dt_acc)}
print(" 判别底座 分层5折CV: Logistic {:.4f} / RF {:.4f} / 决策树 {:.4f}".format(
    cv["lr"], cv["rf"], cv["dt"]))
return disc, cv

```

```

def fit_preweather_models(df):
    import statsmodels.api as sm
    models = {}
    for t in ["高钾", "铅钨"]:
        sub = df[df["类型"] == t].dropna(subset=MODEL_COMPS[t]).copy()
        X = sm.add_constant(sub[MODEL_COMPS[t]])
        m = sm.OLS(sub["Y"], X).fit()
        b = m.params
        w = sub[sub["Y"] > 0][COMPONENTS]
        u = sub[sub["Y"] == 0][COMPONENTS]
        d_sio2 = w["SiO2"].mean() - u["SiO2"].mean()
        ratio = (w.mean() - u.mean()) / d_sio2
        denom = b["SiO2"]
        for c in MODEL_COMPS[t]:
            if c != "SiO2":
                denom += b[c] * ratio[c]
        models[t] = {"coef": b, "ratio": ratio, "denom": denom,
                    "n": len(sub), "R2": m.rsquared}
    print(" 风化还原[{}] R2={:.4f} 反推分母={:.4f} (n={}) ".format(
        t, m.rsquared, denom, len(sub)))
    return models

```

```

def reconstruct(obs_row, hyp_type, models):
    comps = MODEL_COMPS[hyp_type]
    if any(pd.isna(obs_row[c]) or obs_row[c] == 0 for c in comps):
        return None
    m = models[hyp_type]
    dY = WEATHER_Y[hyp_type]
    d_sio2 = dY / m["denom"]
    pre = np.empty(len(COMPONENTS))
    for i, c in enumerate(COMPONENTS):
        v = obs_row[c]

```

```

    r_j = m["ratio"][c]
    if pd.isna(r_j):
        pre[i] = 0.0 if pd.isna(v) else v
    elif pd.isna(v):
        pre[i] = 0.0
    else:
        pre[i] = v - r_j * d_sio2
    if np.any(pre < -0.5) or np.any(pre > 100.5) or pre.sum() > 105.0:
        return None
    return pre

def predict_prob(vec, disc):
    X = np.array(vec, dtype=float).reshape(1, -1)
    p_lr = disc["lr"].predict_proba(disc["scaler"].transform(X))[0, 1]
    p_rf = disc["rf"].predict_proba(X)[0, 1]
    p_dt = disc["dt"].predict_proba(X)[0, 1]
    votes = [TYPE_INV[int(round(p))]] for p in (p_lr, p_rf, p_dt)
    return {"p_lr": float(p_lr), "p_rf": float(p_rf), "p_dt": float(p_dt),
            "votes": votes}

def majority(votes):
    return "铅钨" if votes.count("铅钨") >= 2 else "高钾"

def perturbation_sensitivity(vec, disc):
    rng = np.random.default_rng(RANDOM_STATE)
    vec = np.asarray(vec, dtype=float)
    base = disc["lr"].predict_proba(
        disc["scaler"].transform(vec.reshape(1, -1)))[0, 1]
    out = {}
    for amp in (0.05, 0.10):
        flips = 0
        for _ in range(N_REPEAT):
            vec_p = vec * rng.normal(1.0, amp, size=len(vec))
            p = disc["lr"].predict_proba(
                disc["scaler"].transform(vec_p.reshape(1, -1)))[0, 1]
            if (p > 0.5) != (base > 0.5):
                flips += 1
        out[amp] = flips / N_REPEAT
    return out

def main():
    t_all = time.time()

    stage(1, 5, "构建判别底座（三模型 + 分层 5 折）")
    known = load_known()
    disc, cv = fit_discriminators(known)

    stage(2, 5, "重算风化还原模型（采样点级）")
    sample_df = load_sample_level()
    pw_models = fit_prewether_models(sample_df)

    stage(3, 5, "8 件未知样品 假设-验证 判别")
    unknown = load_unknown()
    rows, sens_rows = [], []
    for _, r in unknown.iterrows():
        wid = r["文物编号"]

```

```

weathered = (r["表面风化"] == "风化")
obs = np.array([r[c] for c in COMPONENTS], dtype=float)

p_direct = predict_prob(obs, disc)
cls_direct = majority(p_direct["votes"])

cand = [(cls_direct, p_direct["p_lr"], "直接判别")]
recons = {}
if weathered:
    for hyp in ["高钾", "铅钡"]:
        pre = reconstruct(r, hyp, pw_models)
        recons[hyp] = pre
        if pre is None:
            print(" [{}] {}假设还原: 关键成分缺失或超出物理范围 → 不可行".format(
                wid, hyp))
            continue
        pp = predict_prob(pre, disc)
        hyp_cls = majority(pp["votes"])
        cand.append((hyp_cls, pp["p_lr"], "{0}假设还原(判为{1})".format(hyp, hyp_cls)))
        print(" [{}] {}假设还原 → 判为{} (P(铅钡)={:.4f}, 投票 {})".format(
            wid, hyp, hyp_cls, pp["p_lr"], "/" + ".join(pp["votes"])))

consist = [c for c in cand if (c[0] == "铅钡") == (c[1] > 0.5)]
pool = consist if consist else cand
final_cls, final_p, basis = max(pool, key=lambda c: abs(c[1] - 0.5))

flip = perturbation_sensitivity(obs, disc)
rows.append({
    "样品": wid, "表面风化": r["表面风化"],
    "直接判别P(铅钡)": round(p_direct["p_lr"], 4),
    "高钾假设还原P(铅钡)":
        round(predict_prob(recons["高钾"], disc)["p_lr"], 4)
        if weathered and recons.get("高钾") is not None else "",
    "铅钡假设还原P(铅钡)":
        round(predict_prob(recons["铅钡"], disc)["p_lr"], 4)
        if weathered and recons.get("铅钡") is not None else "",
    "最终类型": final_cls,
    "置信度": round(abs(final_p - 0.5) * 2, 4),
    "判别依据": basis,
})
sens_rows.append({
    "样品": wid, "表面风化": r["表面风化"], "最终类型": final_cls,
    "±5%扰动翻转率": round(flip[0.05], 4),
    "±10%扰动翻转率": round(flip[0.10], 4),
})
print(" [{}] 表面{} 直接判别={} (P(铅钡)={:.4f}, 投票 {}) → 最终类型={} (依据: {}, 置信度 {:.4f})".format(
    wid, r["表面风化"], cls_direct, p_direct["p_lr"],
    "/" + ".join(p_direct["votes"]), final_cls, basis, abs(final_p - 0.5) * 2))

tab4 = pd.DataFrame(rows)
tab5 = pd.DataFrame(sens_rows)[["样品", "±5%扰动翻转率", "±10%扰动翻转率"]]

stage(4, 5, "敏感性汇总与核对")
print(" 扰动翻转率: 全部为 {} (论文表5)".format(
    "0" if (tab5["±5%扰动翻转率"].astype(str) == "0.0").all()
    and (tab5["±10%扰动翻转率"].astype(str) == "0.0").all() else "非零, 见下表"))
print(tab5.to_string(index=False))

stage(5, 5, "生成论文插图 (PDF) 并写出 第三问.xlsx")

```

```

order = pd.concat([
    tab4[tab4["最终类型"] == "高钾"].sort_values("直接判别P(铅钨)"),
    tab4[tab4["最终类型"] == "铅钨"].sort_values("直接判别P(铅钨)"),
]).reset_index(drop=True)
ids, probs = order["样品"].tolist(), order["直接判别P(铅钨)"].astype(float).tolist()
colors = ["#E8A13D" if t == "高钾" else "#5B8FF9" for t in order["最终类型"]]

fig, ax = plt.subplots(figsize=(7.2, 3.6), dpi=200)
y = np.arange(len(ids))
ax.barh(y, probs, height=0.62, color=colors, edgecolor="none")
ax.axvline(0.5, color="#D9534F", linestyle="--", linewidth=1.2, alpha=0.9)
ax.text(0.5, len(ids) - 0.30, "0.5 判界", color="#D9534F", fontsize=9,
        ha="center", va="bottom")
for yi, p, pid in zip(y, probs, ids):
    ax.text(p + 0.012, yi, "{:.2f}".format(p), va="center", fontsize=9, color="#333")
    ax.text(-0.012, yi, pid, va="center", ha="right", fontsize=9,
            color="#333", fontweight="bold")
ax.set_yticks([]); ax.set_xlim(0, 1.06); ax.set_ylim(-0.6, len(ids) - 0.4)
ax.set_xlabel("判别概率 P(铅钨)", fontsize=10)
ax.set_title("未知样品判别概率（高钾橙色 / 铅钨蓝色）", fontsize=11)
ax.grid(axis="x", linestyle=":", linewidth=0.6, alpha=0.5)
for s in ["top", "right", "left"]:
    ax.spines[s].set_visible(False)
ax.tick_params(axis="x", labelsize=9)
ax.legend(handles=[Patch(color="#E8A13D", label="高钾"),
                   Patch(color="#5B8FF9", label="铅钨")],
          loc="lower right", fontsize=9, frameon=False, ncol=2)
plt.tight_layout()
fig.savefig(os.path.join(FIG_DIR, "fig_q3_prob.pdf"), bbox_inches="tight")
plt.close(fig)
print(" 图4 判别概率 → fig_q3_prob.pdf")

with pd.ExcelWriter(OUT_XLSX, engine="openpyxl") as w:
    tab4.to_excel(w, sheet_name="表4-判别结果", index=False)
    tab5.to_excel(w, sheet_name="表5-翻转率", index=False)

from openpyxl import load_workbook
wb = load_workbook(OUT_XLSX)
for ws in wb.worksheets:
    for col in ws.columns:
        width = max(len(str(c.value)) * 2.2 if c.value is not None else 0
                     for c in col) + 2
        ws.column_dimensions[col[0].column_letter].width = min(width, 40)
wb.save(OUT_XLSX)

print("\n问题 3 完成: 输出 {}; 图 → {} (总耗时 {:.1f}s)".format(
    OUT_XLSX, FIG_DIR, time.time() - t_all))

if __name__ == "__main__":
    main()

```